OnBuch+

SIGNED COURSE • PREMIUM

ICT • UPPER SIXTH | UPPER SIXTH — ICT

CHAPTER 26

ALGORITHMS AND PROGRAMMING

SUGGESTED TIME: 8 PERIODS

IN THIS CHAPTER

This chapter completes the study of algorithms and programming at Advanced Level: selection and loop control structures, flowchart design, dry running of algorithms, and progressive practice in writing code. It prepares learners to analyse a problem, design an algorithmic solution, verify it by hand, and implement it in a programming language — the core skills examined in GCE A-Level ICT.

- By the end of this chapter I can explain the role of control structures (sequence, selection, iteration) in an algorithm.
- By the end of this chapter I can write simple, nested and multiple selection structures (IF…THEN…ELSE, CASE) in pseudocode.
- By the end of this chapter I can write pre-condition loops (WHILE), post-condition loops (REPEAT…UNTIL) and counted loops (FOR) in pseudocode, and choose the appropriate one.
- By the end of this chapter I can draw a correct flowchart using the standard symbols for any given algorithm.
- By the end of this chapter I can dry run (trace) an algorithm with a trace table and predict its output or detect logical errors.
- By the end of this chapter I can translate pseudocode and flowcharts into a high-level programming language (e.g. Python or C).

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Selection control structures
2. Loop control structures
3. Drawing flowcharts
4. Dry running algorithms (trace tables)
5. Writing code and integration
6. Integration activity
7. Methods and worked examples
8. Exercises
9. Solutions to the exercises
10. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain the role of control structures (sequence, selection, iteration) in an algorithm.
- By the end of this chapter I can write simple, nested and multiple selection structures (IF…THEN…ELSE, CASE) in pseudocode.
- By the end of this chapter I can write pre-condition loops (WHILE), post-condition loops (REPEAT…UNTIL) and counted loops (FOR) in pseudocode, and choose the appropriate one.
- By the end of this chapter I can draw a correct flowchart using the standard symbols for any given algorithm.
- By the end of this chapter I can dry run (trace) an algorithm with a trace table and predict its output or detect logical errors.
- By the end of this chapter I can translate pseudocode and flowcharts into a high-level programming language (e.g. Python or C).
- By the end of this chapter I can write, run and debug complete programs combining input/output, selection and loops.
- By the end of this chapter I can solve an authentic problem end-to-end: analysis, algorithm, flowchart, dry run, code and testing.

PREREQUISITES

- Notion of algorithm, pseudocode and variables (Lower Sixth, chapter on introduction to algorithms).

- Data types (integer, real, character, string, boolean) and variable declaration.
- Arithmetic, relational and logical operators and Boolean expressions.
- Basic input/output instructions (READ/INPUT, WRITE/PRINT/DISPLAY) and assignment.
- Sequential structure of a simple algorithm.

1 SELECTION CONTROL STRUCTURES

Every morning, the bursar of a large secondary school in Yaoundé must decide which students have cleared their fees, send reminders to defaulters, and compute totals for hundreds of records. Doing this by hand is slow and error-prone, but a small program can do it in seconds — if someone first thinks through the decisions and repetitions involved. How do we teach a machine to "decide" (if a student has paid, then …) and to "repeat" (for each of the 800 students …)? That is exactly what algorithms with **selection** and **loop** structures allow us to do. In this chapter you will learn to design such solutions with flowcharts, test them by dry running, and finally write real programs. We begin with the structure that gives algorithms the power to choose: the selection control structure.

1.1 The three fundamental control structures

Any algorithm, however complex, is built from only three fundamental ways of organising instructions:

- **Sequence**: instructions are executed one after another, in the order in which they are written.
- **Selection**: the algorithm chooses which block of instructions to execute, depending on whether a condition is true or false.
- **Iteration** (loop): a block of instructions is executed repeatedly, as long as some condition holds.

Sequence alone is not enough. Consider our bursar. A purely sequential algorithm can read a student's record and add an amount to a total — but it cannot decide to send a reminder only to defaulters, nor can it repeat the process for 800 students without writing the same instructions 800 times. Real problems involve choices ("if paid, then mark cleared") and repetitions ("for each student …"). That is why selection and iteration are indispensable. This section studies selection; loops come next.

1.2 Conditions: the heart of selection

Before a machine can choose, it must be able to test something. The test is written as a **condition**.

DEFINITION **Boolean condition**

A **Boolean condition** is an expression that evaluates to exactly one of two values: TRUE or FALSE. For example, `mark >= 50` is TRUE if the variable `mark` holds 72, and FALSE if it holds 30.

Conditions are built using two families of operators.

Relational (comparison) operators. They compare two values:

Operator	Meaning	Example (with `a = 5, b = 8`)
=	equal to	`a = 5` is TRUE
<>	not equal to	`a <> b` is TRUE
<	less than	`a < b` is TRUE
>	greater than	`a > b` is FALSE
<=	less than or equal to	`a <= 5` is TRUE
>=	greater than or equal to	`b >= 9` is FALSE

Logical operators. They combine conditions into compound conditions:

- `AND`: TRUE only if both conditions are TRUE. Example: `(mark >= 0) AND (mark <= 20)` checks that a mark is valid.
- `OR`: TRUE if at least one condition is TRUE. Example: `(day = "Saturday") OR (day = "Sunday")`.
- `NOT`: reverses the truth value. `NOT paid` is TRUE exactly when `paid` is FALSE.

We shall also use the arithmetic operator `MOD`, which gives the remainder of an integer division: `17 MOD 5 = 2` because $17 = 3 \times 5 + 2$. It is the standard tool for testing divisibility: `n MOD 2 = 0` means "`n` is even".

WARNING **Trap with AND and OR**

To test whether `mark` lies between 50 and 100 inclusive, write `(mark >= 50) AND (mark <= 100)`. Writing `50 <= mark <= 100` (mathematical shorthand) or using `OR` instead of `AND` are classic errors: with `OR`, every number satisfies at least one side, so the test is always TRUE.

1.3 Simple selection: IF … THEN … END IF

The simplest decision is: do something only if a condition holds; otherwise do nothing special.

DEFINITION **Simple selection**

The **simple selection** structure executes a block of instructions only when its condition is TRUE; if the condition is FALSE, the block is skipped and execution continues after the `END IF`.

In pseudocode:

```
IF feesPaid >= feesDue THEN
    status <- "CLEARED"
END IF
DISPLAY status
```

The flow of execution is: evaluate the condition; if TRUE, run the indented block, then continue; if FALSE, jump directly to the instruction after `END IF`. Note the **indentation**: the block inside the `IF` is shifted to the right so the structure is visible at a glance. The `END IF` keyword marks precisely where the conditional block finishes — never omit it. (Some textbooks write `ENDIF` as a single word; both forms are accepted at the GCE, but be consistent.)

EXEMPLE **A gentle fine**

A cybercafé in Buea charges a penalty of 500 FCFA only if a customer exceeds the paid time:

```
IF minutesUsed > minutesPaid THEN
bill <- bill + 500
END IF
```

If `minutesUsed = 40` and `minutesPaid = 60`, the condition is FALSE, so `bill` is left unchanged. Nothing "bad" happens — the block is simply skipped.

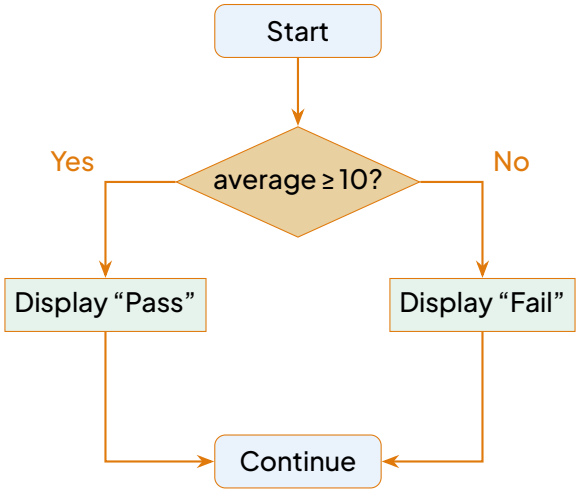
1.4 Alternative selection: IF … THEN … ELSE … END IF

Often we must choose between two actions: one when the condition is TRUE, another when it is FALSE.

DEFINITION **Alternative selection**

The **alternative selection** structure `IF condition THEN block1 ELSE block2 END IF` executes `block1` when the condition is TRUE and `block2` when it is FALSE. Exactly one of the two blocks is executed — never both, never none.

```
IF average >= 10 THEN
    DISPLAY "Pass"
ELSE
    DISPLAY "Fail"
END IF
```



Flowchart of an IF…THEN…ELSE structure: the diamond is the decision, with exactly two branches labelled Yes and No.

The diamond shape is the universal flowchart symbol for a decision. Observe that the two branches always rejoin afterwards: selection changes which instructions run, not the fact that the algorithm continues.

1.5 Nested selection

An `IF` may contain another `IF` inside it: this is a **nested selection**. We use it when a second decision only makes sense after a first one has been taken.

```
IF paid = TRUE THEN
    IF balance > 0 THEN
        DISPLAY "Part payment: still owe ", balance
    ELSE
        DISPLAY "Fully cleared"
    END IF
ELSE
    DISPLAY "Send reminder"
END IF
```

Each `IF` has its own `END IF`, and indentation shows which `ELSE` belongs to which `IF`. This matters because of the famous **dangling else** problem: in `IF A THEN IF B THEN X ELSE Y`, does the `ELSE` pair with the first `IF` or the second? In well-written pseudocode, indentation and `END IF` markers remove all ambiguity — the `ELSE` always belongs to the nearest unmatched `IF` above it.

WARNING **Mismatched IF/ELSE pairs**

Forgetting an `END IF` or misaligning an `ELSE` in nested selections does not just look untidy:

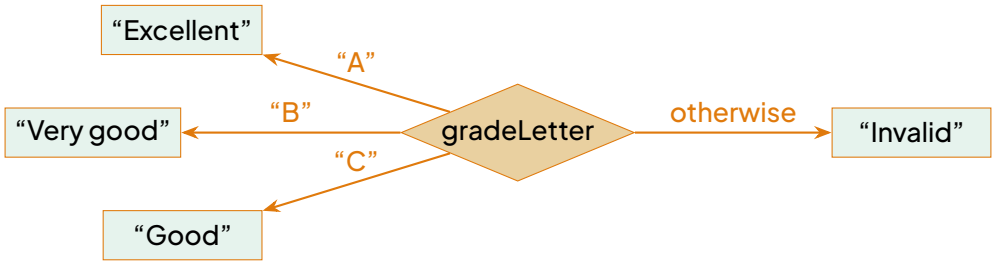
it changes the logic of the program. An `ELSE` that you meant for the outer `IF` may silently attach itself to the inner one, and the algorithm will treat fully-paid students as defaulters. Always count your `IF`s and `END IF`s, and indent rigorously.

1.6 Multiple selection: the CASE structure

When one variable must be tested against several discrete values, nesting many `IF…ELSE IF` becomes heavy. The **CASE** (or `SWITCH`) structure is designed for this:

```
CASE gradeLetter OF
    "A" : DISPLAY "Excellent"
    "B" : DISPLAY "Very good"
    "C" : DISPLAY "Good"
    "D" : DISPLAY "Fair"
    "E" : DISPLAY "Weak"
    OTHERWISE : DISPLAY "Invalid grade"
END CASE
```

The value of `gradeLetter` is compared with each listed case; the matching block runs, and the optional `OTHERWISE` (default) branch catches every value not listed. A `CASE` is exactly equivalent to a chain of nested `IF`s testing equality (`IF gradeLetter = "A" THEN …ELSE IF gradeLetter = "B" THEN …`), but it is shorter and clearer.



A CASE structure: one decision point, several branches, one per value (only some branches shown).

METHOD **Choosing between IF… ELSE and CASE**

1. Look at what is being tested.
2. If it is one variable compared to several discrete values (a letter, a menu option, a day number), use `CASE`.
3. If the tests involve ranges (`mark >= 80`), several different variables, or compound con-

ditions with `AND`/`OR`, use nested `IF…ELSE IF`.

4. Remember the two are convertible: a `CASE` is a chain of equality tests; but a range-based nested `IF` generally cannot be written as a simple `CASE`.

1.7 Worked examples

WORKED EXAMPLE 1 **Largest of three numbers**

Write a pseudocode algorithm that reads three distinct numbers `a`, `b`, `c` and displays the largest.

SOLUTION

Solution. Compare `a` with `b` first, then compare the winner with `c`:

```
READ a, b, c
IF a > b THEN
max <- a
ELSE
max <- b
END IF
IF c > max THEN
max <- c
END IF
DISPLAY "Largest = ", max
```

Check with $a = 12$, $b = 7$, $c = 20$: first `max <- 12`; then $20 > 12$ is TRUE, so `max <- 20`. Output: `Largest = 20`. Correct.

WORKED EXAMPLE 2 **Grading a mark**

A school grades a mark out of 100 as follows: A for 80 and above, B for 70–79, C for 60–69, D for 50–59, E below 50. Write the pseudocode and draw the decision tree.

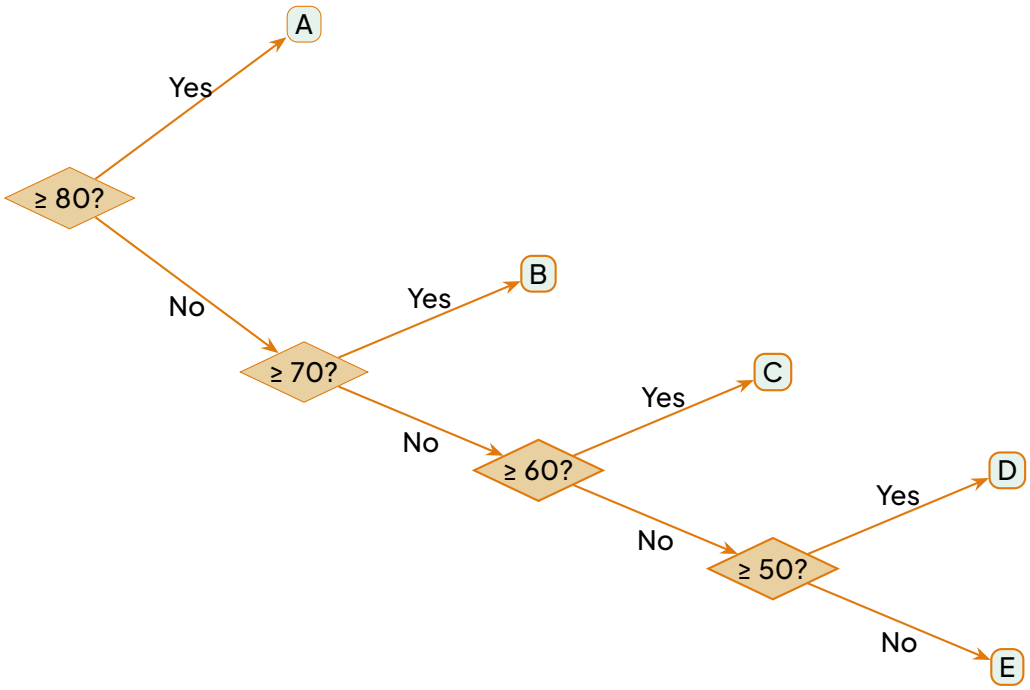
SOLUTION

Solution. Test the thresholds from the top down; once a test succeeds, the rest are skipped:

```
READ mark
IF mark >= 80 THEN
grade <- "A"
ELSE IF mark >= 70 THEN
grade <- "B"
```

```
ELSE IF mark >= 60 THEN
grade <- "C"
ELSE IF mark >= 50 THEN
grade <- "D"
ELSE
grade <- "E"
END IF
DISPLAY grade
```

The order is essential: because the tests are chained, a mark of 75 fails `>= 80` but passes `>= 70`, and we never need to write `(mark >= 70) AND (mark < 80)` — the upper bound is guaranteed by the previous failed test.



Decision tree for the grading algorithm: each diamond is a threshold test, each leaf a grade.

WORKED EXAMPLE 3 **Leap year**

A year is a leap year if it is divisible by 4, except years divisible by 100, unless they are also divisible by 400. Write the condition as one compound Boolean expression and as a nested selection.

SOLUTION

Solution. As a compound condition:

```
leap <- ((year MOD 400) = 0) OR (((year MOD 4) = 0) AND ((year MOD 100) <> 0))
```

As nested selection:

```
IF (year MOD 100) = 0 THEN
IF (year MOD 400) = 0 THEN leap <- TRUE ELSE leap <- FALSE END IF
ELSE
IF (year MOD 4) = 0 THEN leap <- TRUE ELSE leap <- FALSE END IF
END IF
```

Test: 2000 gives TRUE (divisible by 400); 1900 gives FALSE (divisible by 100 but not 400); 2024 gives TRUE.

EXEMPLE **Fee status of a student**

Back to our bursar. For each student, `feesDue` and `feesPaid` are known:

```
IF feesPaid >= feesDue THEN
DISPLAY name, " : CLEARED"
ELSE IF feesPaid > 0 THEN
DISPLAY name, " : PART PAYMENT, balance = ", feesDue - feesPaid
ELSE
DISPLAY name, " : DEFAULTER --- send reminder"
END IF
```

Three mutually exclusive situations, handled by one nested selection. Notice that the conditions do not overlap and cover every possibility.

1.8 Common logical errors and exam advice

WARNING **Classic mistakes with selection**

- Confusing assignment and comparison. `IF mark = 50` is a test; `mark <- 50` is an assignment. Mixing them up changes data instead of testing it.
- Missing `END IF`. The compiler or examiner cannot guess where your block ends; everything after may be swallowed into the `IF`.
- Overlapping or incomplete conditions. If you test `mark >= 70` for B before `mark >= 80` for A, every A student is labelled B. Order range tests carefully, from the most restrictive to the least, and make sure every possible input falls into exactly one branch.
- Wrong logical operator. Using `OR` where `AND` is needed makes a condition almost always TRUE.

WARNING **GCE exam tip**

In the Advanced Level paper you are frequently asked to (a) convert a nested IF into a CASE and vice versa, and (b) state the exact output of a selection structure for given input values. For (a), remember: `CASE` works only for equality tests on one variable. For (b), evaluate the conditions in order, follow the single branch that applies, and write the output exactly as the `DISPLAY` instructions produce it — examiners award marks for the precise values, not for a description.

KEY POINTS **Key points**

- Algorithms are built from three control structures: **sequence**, **selection**, **iteration**.
- A **Boolean condition** evaluates to TRUE or FALSE, using relational operators (`=`, `<>`, `<`, `>`, `<=`, `>=`) and logical operators (`AND`, `OR`, `NOT`).
- `IF …THEN …END IF` executes a block only when the condition is TRUE; `IF …THEN …ELSE …END IF` chooses between two blocks, exactly one of which runs.
- Selections can be **nested**; indentation and matched `END IF`s prevent the dangling-else ambiguity.
- `CASE` selects among several discrete values of one variable and is equivalent to a chain of equality-testing `IF`s; use nested `IF` for ranges.

EXERCISE 1 **Practice**

1. Write pseudocode that reads the temperature and displays “Freezing” if it is below 0, “Cold” from 0 to 15, “Warm” from 16 to 30, and “Hot” above 30.
2. Rewrite the following as a CASE structure: `IF option = 1 THEN DISPLAY "Deposit" ELSE IF option = 2 THEN DISPLAY "Withdrawal" ELSE IF option = 3 THEN DISPLAY "Balance" ELSE DISPLAY "Invalid" END IF`.
3. For the fee-status algorithm above, state the output when `feesDue = 50000` and `feesPaid = 30000`.
4. A student wrote `IF (age > 18) OR (age = 18) THEN DISPLAY "Adult"`. Simplify the condition and explain the simplification.

SOLUTION **Exercise 1**

```
READ t
IF t < 0 THEN DISPLAY "Freezing"
ELSE IF t <= 15 THEN DISPLAY "Cold"
ELSE IF t <= 30 THEN DISPLAY "Warm"
ELSE DISPLAY "Hot" END IF
```

The chained tests make the ranges mutually exclusive; order from the lowest threshold upward (or highest downward).

SOLUTION **Exercise 2**

```
CASE option OF
1 : DISPLAY "Deposit"
2 : DISPLAY "Withdrawal"
3 : DISPLAY "Balance"
OTHERWISE : DISPLAY "Invalid"
END CASE
```

This is valid because one variable (`option`) is tested for equality against discrete values.

SOLUTION **Exercise 3**

`feesPaid >= feesDue` is FALSE (30000 < 50000); `feesPaid > 0` is TRUE, so the second branch runs. Output: the student's name followed by `: PART PAYMENT, balance = 20000`.

SOLUTION **Exercise 4**

`(age > 18) OR (age = 18)` is exactly `age >= 18`: "greater than or equal to" already means "strictly greater or equal". The simplified condition is shorter, clearer and less error-prone.

Selection lets an algorithm choose — but our bursar still has to process 800 students one by one. Writing the same instructions 800 times is unthinkable. What we need now is a structure that repeats a block of instructions automatically: the loop control structure, which is the subject of the next section.

2 LOOP CONTROL STRUCTURES

In the previous section we saw how **selection control structures** allow a program to choose between alternative paths. Many real-world problems, however, require repeating the same set of actions many times: printing a receipt for every customer in a queue at a Mobile Money agent in Douala, calculating the total sales for each day of the month for a shop in Bamenda, or asking a user for a valid PIN until they enter it correctly on an MTN or Orange USSD menu. Writing the same code over and over is tedious, error-prone and impossible when the number of repetitions is not known in advance. This is where **loop control structures** (also called **iteration** or **repetition** structures) come in.

2.1 Why loops? Components of a loop

A loop repeats a block of instructions — the **loop body** — while a certain condition holds or until a condition becomes true. Every well-formed loop has four logical components:

1. **Initialisation:** setting up variables before the first iteration (e.g. a counter to 0, an accumulator to 0).
2. **Condition (test):** a Boolean expression that decides whether the body should execute again.
3. **Body:** the statements that are repeated.
4. **Update (step):** modification of the loop variable(s) inside the body so that the condition eventually becomes false.

Missing any of these — especially the update — leads to an **infinite loop**, a classic bug that hangs the program.

DEFINITION **Loop (iteration/repetition) control structure**

A loop control structure repeats a block of instructions (the loop body) while a condition remains true (pre-condition loop) or until a condition becomes true (post-condition loop), or for a predetermined number of iterations (counted loop).

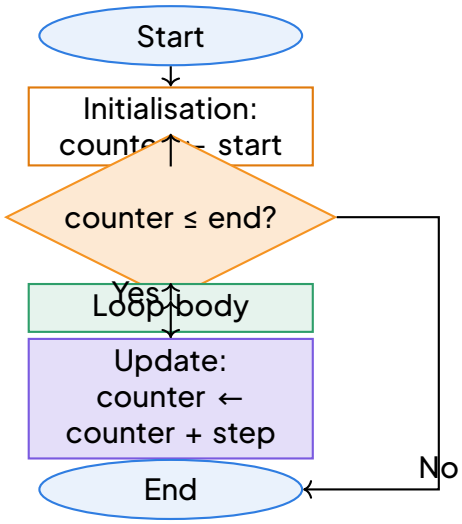
2.2 Counted loop: FOR

When the number of repetitions is known before the loop starts, we use a **FOR** loop (counted loop). The generic syntax in pseudocode is:

```
FOR counter $\leftarrow$ start TO end [STEP s]
  ... body ...
END FOR
```

- `counter` is a variable that takes successive values.
- `start` is the initial value, `end` is the final value (inclusive).

- `STEP s` is optional; if omitted, `s = 1`. It can be negative to count down.
- The number of iterations is $\left\lfloor \frac{\text{end}-\text{start}}{s} \right\rfloor + 1$ (provided the progression reaches `end` with positive step and start ≤ end).



Flowchart of a FOR loop: initialisation, test, body, update cycle.

EXEMPLE **Counting up and down**

Count up from 1 to 10:

```
FOR i $\leftarrow$ 1 TO 10 STEP 1
  DISPLAY i
END FOR
```

Count down from 10 to 1:

```
FOR i $\leftarrow$ 10 TO 1 STEP -1
  DISPLAY i
END FOR
```

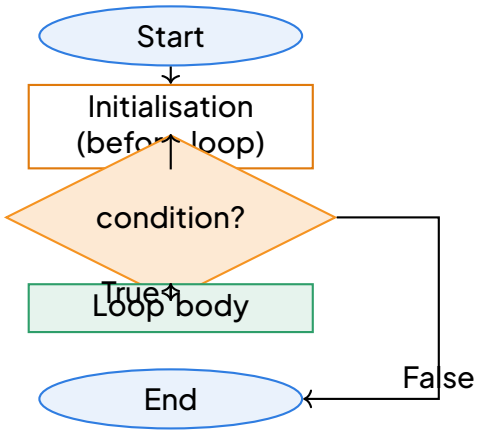
Both loops execute exactly 10 iterations.

2.3 Pre-condition loop: WHILE

When the number of iterations is not known in advance and depends on a condition that is tested before each iteration, we use a **WHILE** loop.

```
WHILE condition DO
  ... body ...
END WHILE
```

The condition is evaluated **before** the first execution of the body. If it is false initially, the body never executes (zero iterations).



WHILE loop: test before body (may execute zero times).

PROPERTY **WHILE loop execution count**

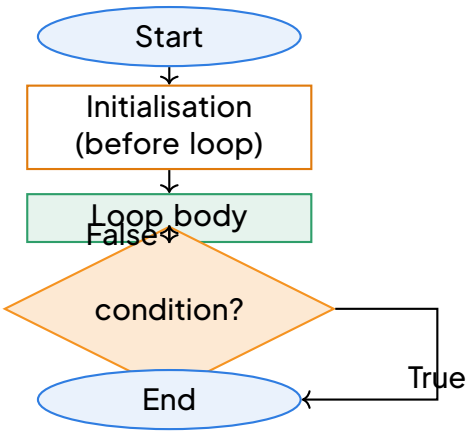
A WHILE loop executes its body zero or more times. It executes zero times if the condition is false at the very first test.

2.4 Post-condition loop: REPEAT … UNTIL

When the body must execute at least once before the condition is checked, we use a **REPEAT … UNTIL** loop.

```
REPEAT
  ... body ...
UNTIL condition
```

The condition is evaluated **after** the body. The loop stops when the condition becomes **true** (note the difference: WHILE continues while true, REPEAT continues until true).



REPEAT … UNTIL loop: test after body (executes at least once).

PROPERTY **REPEAT … UNTIL execution count**

A REPEAT … UNTIL loop executes its body one or more times. It always executes at least once because the test occurs after the first pass.

2.5 Choosing the right loop

Loop type	When to use	Test position	Minimum iterations
FOR	Number of iterations known in advance (counting)	Implicit (before each)	0 (if start > end and step positive)
WHILE	Condition-controlled, may run zero times (e.g. sentinel input validation)	Before body	0
REPEAT … UNTIL	Condition-controlled, must run at least once (e.g. menu display, password prompt)	After body	1

METHOD **Converting FOR to WHILE (examinable)**

To rewrite a FOR loop as a WHILE loop:

1. Initialise the counter before the loop.
2. Use the FOR loop's termination condition as the WHILE condition.

3. Place the update (increment/decrement) as the last statement inside the WHILE body.

Example:

```
FOR i $\leftarrow$ 1 TO 10 STEP 1
  DISPLAY i
END FOR
```

becomes

```
i $\leftarrow$ 1
WHILE i $\le$ 10 DO
  DISPLAY i
  i $\leftarrow$ i + 1
END WHILE
```

2.6 Accumulators and counters inside loops

Loops are the natural place to compute aggregates over a series of values. Two key patterns:

- **Counter:** a variable incremented by 1 each iteration to count occurrences.
- **Accumulator:** a variable that accumulates a running total (sum, product, etc.).

Both must be **initialised before the loop** (usually to 0 for sum/count, 1 for product).

WORKED EXAMPLE 4 **Sum of first N integers**

Problem: Write an algorithm that reads a positive integer N and computes the sum $1 + 2 + \cdots + N$.
Solution (FOR loop):

```
DISPLAY "Enter N: "
READ N
sum $\leftarrow$ 0
FOR i $\leftarrow$ 1 TO N STEP 1
  sum $\leftarrow$ sum + i
END FOR
DISPLAY "Sum = ", sum
```

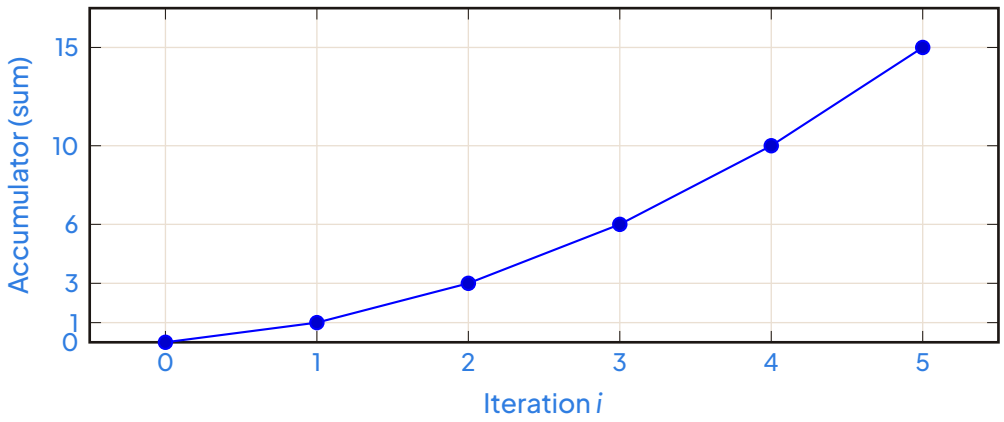
Trace for N = 5:

Iteration	i	sum
Initial	–	0
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

Output: Sum = 15

SOLUTION

Detailed solution: The accumulator `sum` starts at 0. Each iteration adds the current value of `i` to `sum`. After 5 iterations, `sum` holds 1 + 2 + 3 + 4 + 5 = 15. This pattern (initialise accumulator, loop, update accumulator) is fundamental for all aggregation tasks.



Accumulator growth over iterations (sum of 1 to 5).

WORKED EXAMPLE 5 **Average of marks entered until –1 (sentinel)**

Problem: Students enter their marks one by one. The input ends when –1 is typed. Compute the average (ignore the –1).
Solution (WHILE loop with sentinel):

```
sum $\leftarrow$ 0
count $\leftarrow$ 0
DISPLAY "Enter mark (-1 to stop): "
READ mark
WHILE mark $\neq$ -1 DO
```

```
  sum $\leftarrow$ sum + mark
  count $\leftarrow$ count + 1
  DISPLAY "Enter mark (-1 to stop): "
  READ mark
END WHILE
IF count > 0 THEN
  average $\leftarrow$ sum / count
  DISPLAY "Average = ", average
ELSE
  DISPLAY "No marks entered."
END IF
```

Key points:

- The first READ (priming read) is before the loop.
- The condition checks the sentinel before adding to sum/count.
- The update READ is at the bottom of the loop body.
- Guard against division by zero.

SOLUTION

Detailed solution: This is the classic sentinel-controlled loop pattern. The priming read gets the first value. The WHILE test excludes the sentinel (-1) from processing. The read at the bottom fetches the next value for the next test. The final IF prevents division by zero if the user enters -1 immediately.

WORKED EXAMPLE 6 **Counting pass/fail in a class of 50**

Problem: For a class of 50 students, read each mark and count how many passed (mark ≥ 50) and how many failed.
Solution (FOR loop with counters):

```
pass $\leftarrow$ 0
fail $\leftarrow$ 0
FOR student $\leftarrow$ 1 TO 50 STEP 1
  DISPLAY "Enter mark for student ", student, ": "
  READ mark
  IF mark >= 50 THEN
    pass $\leftarrow$ pass + 1
  ELSE
    fail $\leftarrow$ fail + 1
  END IF
END FOR
```

```
DISPLAY "Passed: ", pass, " Failed: ", fail
```

SOLUTION

Detailed solution: The FOR loop guarantees exactly 50 iterations. Two counters (`pass`, `fail`) are initialised to 0. Each iteration reads one mark and uses a selection structure to increment the appropriate counter. This combines selection (Lesson 80) and iteration (Lesson 81) — a typical integration task.

2.7 Nested loops

A **nested loop** places one loop completely inside the body of another. The inner loop runs to completion for each single iteration of the outer loop. Total iterations = (outer iterations) × (inner iterations).

EXEMPLE **Multiplication table 1..5**

```
FOR i $\leftarrow$ 1 TO 5 STEP 1
  FOR j $\leftarrow$ 1 TO 5 STEP 1
    DISPLAY i, " x ", j, " = ", i*j
  END FOR
  DISPLAY ""  // blank line after each row
END FOR
```

Outer loop runs 5 times; inner loop runs 5 times each → 25 executions of the display statement.

EXEMPLE **Processing a class list per subject**

Suppose we have 3 subjects and 4 students. We want to read marks for each student in each subject.

```
FOR subject $\leftarrow$ 1 TO 3 STEP 1
  DISPLAY "Subject ", subject
  FOR student $\leftarrow$ 1 TO 4 STEP 1
    DISPLAY " Student ", student, " mark: "
    READ mark
    // process mark (e.g. accumulate per subject)
  END FOR
END FOR
```

2.8 Infinite loops and loop termination

An **infinite loop** occurs when the loop condition never becomes false. Common causes:

- Forgetting to update the loop variable inside the body.
- Updating the variable in the wrong direction (e.g. incrementing when the test expects a decrease).
- A logic error in the condition itself.

WARNING **Forgetting the update — infinite loop**

Wrong:

```
i $\leftarrow$ 1
WHILE i $\le$ 10 DO
  DISPLAY i
  // missing i $\leftarrow$ i + 1
END WHILE
```

This prints 1 forever. Always ensure the update statement is present and moves the variable toward the terminating condition.

WARNING **Off-by-one errors in FOR bounds**

Wrong: Wanting 10 iterations but writing `FOR i $\leftarrow$ 0 TO 10` (gives 11 iterations: 0,1,…,10). Correct: `FOR i $\leftarrow$ 1 TO 10` or `FOR i $\leftarrow$ 0 TO 9`. Always check: number of iterations = end − start + 1 (for step 1).

KEY POINTS **Exam tips for GCE Advanced Level**

- Know the three loop types (FOR, WHILE, REPEAT…UNTIL) and their flowchart symbols.
- Be able to state the number of iterations of a FOR loop given start, end, step.
- Be able to rewrite a FOR loop as a WHILE loop (and vice-versa) — this is a classic exam question.
- Trace tables (dry runs) for loops are frequently asked: show the values of all variables at each iteration.
- Nested loops: calculate total inner loop executions = outer iterations × inner iterations.
- Sentinel-controlled loops: remember the priming read and the read at the bottom of

the loop.

- Initialise accumulators (0 for sum, 1 for product) and counters (0) before the loop.

2.9 Worked examples: factorial and more

WORKED EXAMPLE 7 **Factorial of N (N!)**

Problem: Compute $N! = 1 \times 2 \times \cdots \times N$ for a given non-negative integer N. (0! = 1)
Algorithm (FOR loop with product accumulator):

```
DISPLAY "Enter N: "
READ N
fact $\leftarrow$ 1
FOR i $\leftarrow$ 1 TO N STEP 1
  fact $\leftarrow$ fact * i
END FOR
DISPLAY N, "! = ", fact
```

Trace for N = 4:

Iteration	i	fact
Initial	-	1
1	1	1
2	2	2
3	3	6
4	4	24

Output: 4! = 24

SOLUTION

Detailed solution: The accumulator `fact` starts at 1 (multiplicative identity). For N=0, the loop condition `i $\le$ 0` is false initially, so the body never executes and `fact` remains 1, correctly giving 0! = 1. For N>0, each iteration multiplies by the next integer.

WORKED EXAMPLE 8 **Rewriting a FOR loop as WHILE (exam style)**

Given:

```
FOR k $\leftarrow$ 5 TO 1 STEP -1
  DISPLAY k
```

```
END FOR
```

Number of iterations: 5 (values 5,4,3,2,1).
Equivalent WHILE loop:

```
k $\leftarrow$ 5
WHILE k $\ge$ 1 DO
  DISPLAY k
  k $\leftarrow$ k - 1
END WHILE
```

SOLUTION

Detailed solution: The FOR loop counts down from 5 to 1 inclusive. The equivalent WHILE initialises k=5, tests k>=1 (the continuation condition), and decrements k at the end of the body. Note the condition is `k >= 1` not `k > 1` because the FOR loop includes 1.

2.10 Integration: a complete scenario

Imagine a small program for a teacher at **Government Bilingual High School Molyko** to compute the class average for a test, but only for students who actually sat the exam (absent students are marked with -1). The class has 40 students.

WORKED EXAMPLE 9 **Class average with absent students**

```
sum $\leftarrow$ 0
count $\leftarrow$ 0
FOR student $\leftarrow$ 1 TO 40 STEP 1
  DISPLAY "Mark for student ", student, " (-1 if absent): "
  READ mark
  IF mark $\neq$ -1 THEN
    sum $\leftarrow$ sum + mark
    count $\leftarrow$ count + 1
  END IF
END FOR
IF count > 0 THEN
  average $\leftarrow$ sum / count
  DISPLAY "Class average (present students): ", average
ELSE
  DISPLAY "No student present."
END IF
```

SOLUTION

Detailed solution: This combines a counted loop (FOR), a selection (IF), accumulators (`sum`, `count`) and a guard against division by zero — exactly the kind of integration activity the syllabus expects (Lesson 87). The sentinel value -1 is checked inside the loop, so absent students are excluded from both sum and count.

EXERCISE 2 **Practice**

1. Write a FOR loop that displays the even numbers from 2 to 20 inclusive.
2. Rewrite the loop in (1) as a WHILE loop.
3. Design an algorithm using a REPEAT…UNTIL loop that asks the user for a positive integer and keeps asking until the user enters a value > 0.
4. A loop runs: `FOR x $\leftarrow$ 3 TO 15 STEP 3`. How many iterations? What values does x take?
5. Trace the following algorithm for input 6:

```
READ N
p $\leftarrow$ 1
FOR i $\leftarrow$ 2 TO N STEP 1
  p $\leftarrow$ p * i
END FOR
DISPLAY p
```

   What does this algorithm compute?

SOLUTION **Exercise 1**

1. `FOR i $\leftarrow$ 2 TO 20 STEP 2 \quad DISPLAY i \quad END FOR`
2. `i $\leftarrow$ 2 \quad WHILE i $\le$ 20 DO \quad DISPLAY i \quad i $\leftarrow$ i + 2 \quad END WHILE`
3. `REPEAT \quad DISPLAY "Enter positive integer: " \quad READ n \quad UNTIL n > 0`
4. Iterations: (15 − 3)/3 + 1 = 5. Values: 3, 6, 9, 12, 15.
5. Trace: N=6, p=1 initially. i=2: p=2; i=3: p=6; i=4: p=24; i=5: p=120; i=6: p=720. Output

720. This computes $N!$ (factorial) but starting from 2 (since 1! = 1, it's correct for N≥1).

DID YOU KNOW? **Loops in everyday Cameroon tech**

When you dial `*123#` on MTN or `#123#` on Orange to check your balance, the USSD menu system uses loops: it repeats the menu display (REPEAT…UNTIL) until you choose "Exit". Mobile Money transaction validation often uses a WHILE loop to retry a failed PIN entry up to 3 times (a counted loop nested inside a condition-controlled loop). The Cameroon GCE Board's own online registration system processes thousands of candidates by looping through a database record set — a classic FOR-EACH style iteration.

3 DRAWING FLOWCHARTS

In the previous section we mastered **loop control structures** — `WHILE`, `REPEAT` and `FOR` — and saw how they let an algorithm repeat a block of instructions until a condition becomes false. We wrote pseudocode for summing numbers, finding averages and processing sentinel-controlled input. But pseudocode is only one way to represent an algorithm. In examinations and in professional documentation, a **flowchart** is often required because it gives a visual, language-independent picture of the logic. This section teaches you to draw correct, complete flowcharts for any algorithm built from sequence, selection and iteration.

3.1 Purpose of a flowchart

A **flowchart** is a diagram that represents the steps of an algorithm using standardised symbols connected by arrows. It serves three main purposes:

- **Communication:** A flowchart can be understood by programmers, managers and clients regardless of the programming language they use.
- **Documentation:** It provides a permanent record of the algorithm's logic for future maintenance.
- **Design and debugging:** Drawing a flowchart forces you to think through every decision and loop before writing code; errors in logic often become obvious on paper.

DEFINITION **Flowchart**

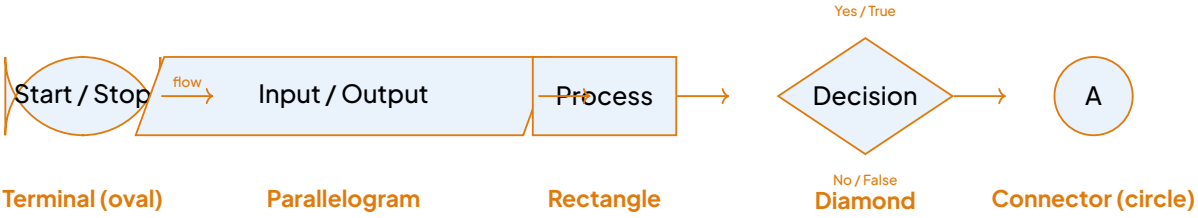
A flowchart is a graphical representation of an algorithm that uses standardised symbols (oval, parallelogram, rectangle, diamond, circle) connected by directed flow lines to show the sequence of operations, decisions and data movements.

In the Cameroon GCE Advanced Level examination, you may be asked to:

- Draw a flowchart from a given pseudocode or problem statement.
- Interpret a given flowchart: state what it does, complete a trace table, or give the output for specific inputs.
- Convert a flowchart back into pseudocode.

3.2 Standard flowchart symbols

The International Organization for Standardization (ISO 5807) and the Cameroon GCE Board expect the following symbols. Memorise their shapes and uses.



Standard flowchart symbols with their names and typical uses.

Symbol	Name	Meaning
Oval (rounded rectangle)	Terminal	Start or Stop of the algorithm
Parallelogram	Input / Output	Reading data (Input) or displaying results (Output)
Rectangle	Process	Calculation, assignment, or any internal operation
Diamond	Decision	A condition with exactly two outcomes: Yes/True and No/False
Circle	Connector	Links broken flow lines across pages or distant parts (labelled A, B, …)
Arrowed line	Flow line	Shows direction of control (always top-to-bottom or left-to-right)

WARNING **Symbol confusion — common mistake**

Never use a rectangle for a decision or a diamond for a process. Examiners penalise this heavily. Also do not confuse the parallelogram (input/output) with the rectangle (process). Example: `Read mark` goes in a parallelogram; `total $\leftarrow$ total + mark` goes in a rectangle.

3.3 Rules for drawing a correct flowchart

METHOD **Drawing a flowchart — step by step**

- Start with a Terminal oval labelled “Start”.
- Flow top-to-bottom (preferred) or left-to-right. Keep the diagram neat and spaced.
- Use the correct symbol for each step (see table above).
- Draw arrows on every flow line. A line without an arrow is ambiguous.
- Every decision diamond must have exactly two labelled exits: “Yes” (or “True”) and “No” (or “False”). Write the labels on the arrows.
- No dangling paths: every path must eventually reach a “Stop” terminal.
- Use connectors (circles with letters) only when the flowchart is too large for one page or when lines would cross confusingly. Matching pairs (A to A, B to B) must be clearly labelled.
- End with a Terminal oval labelled “Stop”.

WARNING **Missing arrows and unlabelled branches**

A flowchart without arrows on flow lines is incomplete and loses marks. A decision diamond with only one exit, or with exits not labelled Yes/No, is ambiguous and loses marks. Always draw both branches and label them.

3.4 Flowcharting selection structures

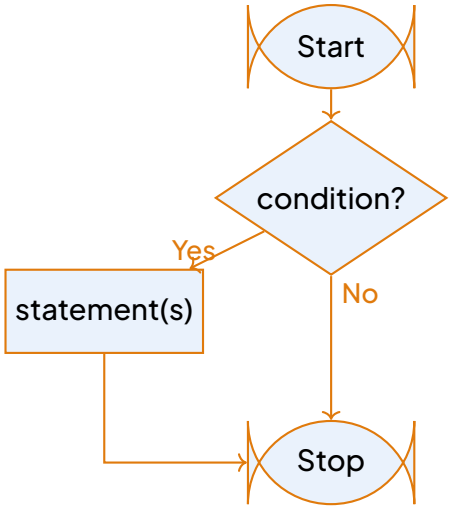
Selection means choosing between alternative paths. The three common patterns are `IF`, `IF...ELSE` and `CASE` (multiple selection). Nested `IF` is simply an `IF` inside another `IF`.

3.5 Simple IF (one-way selection)

Pseudocode:

```
IF condition THEN
\quad statement(s)
    END IF
```

Flowchart pattern:



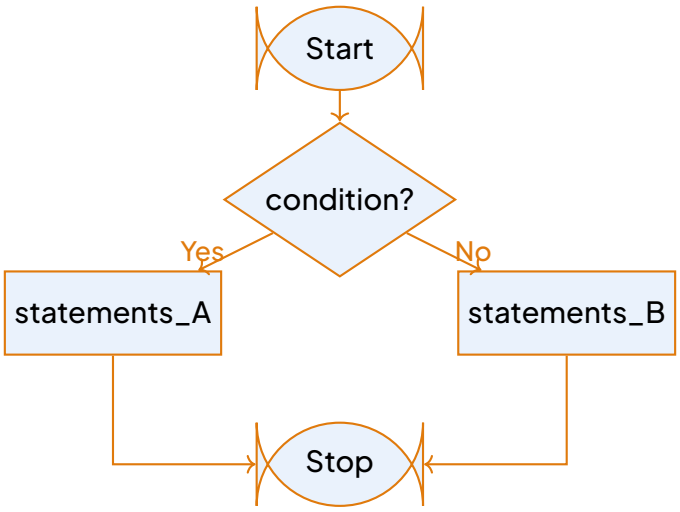
Flowchart for a simple IF (one-way selection).

3.6 IF…ELSE (two-way selection)

Pseudocode:

```
IF condition THEN
\quad statements_A
      ELSE
\quad statements_B
    END IF
```

Flowchart pattern:



Flowchart for IF…ELSE (two-way selection). Both branches rejoin after the statements.

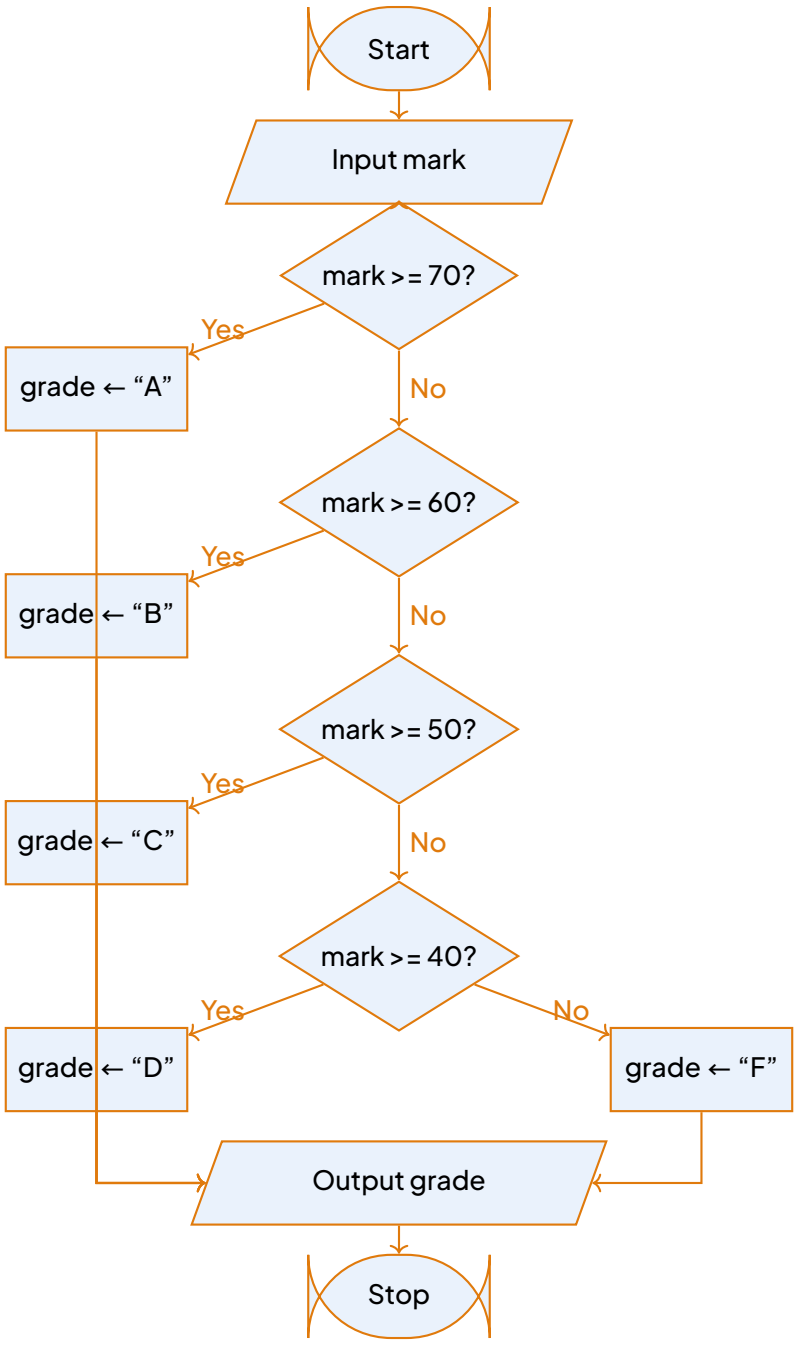
3.7 Nested IF and CASE (multiple selection)

A `CASE` (or `SWITCH`) structure tests a single variable against several constant values. In a flowchart it is drawn as a **chain of decision diamonds**, each testing one value, with a final "Else" branch for unmatched values.

Pseudocode for grading a mark (0–100):

```
                CASE mark OF
\quad 70..100:  grade $\leftarrow$ ``A''
 \quad 60..69:  grade $\leftarrow$ ``B''
 \quad 50..59:  grade $\leftarrow$ ``C''
 \quad 40..49:  grade $\leftarrow$ ``D''
  \quad ELSE: grade $\leftarrow$ ``F''
                  END CASE
```

Flowchart (complete worked example):



Complete flowchart for grading a mark using a chain of decisions (CASE pattern).

KEY POINTS **CASE as a chain of diamonds**

A CASE structure is flowcharted as a vertical chain of decision diamonds. Each diamond tests one condition (e.g., `mark >= 70?`). The “Yes” branch goes to the corresponding action rectangle, which then jumps to the common exit (here `Output grade`). The “No”

branch goes to the next diamond. The final "No" branch goes to the `ELSE` action.

3.8 Flowcharting loop structures

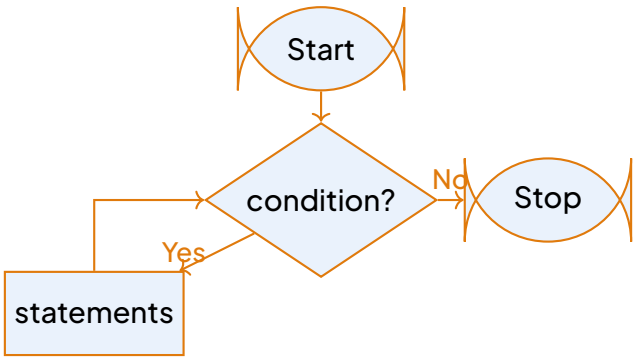
Loops repeat a block of instructions. The two fundamental patterns are **pre-test** (`WHILE`) and **post-test** (`REPEAT...UNTIL`). A `FOR` loop is a counted pre-test loop with initialisation and increment built in.

3.9 WHILE loop (pre-test)

Pseudocode:

```
WHILE condition DO
\quad statements
    END WHILE
```

Flowchart pattern — the decision comes **before** the process, and the "Yes" branch loops back to the decision:



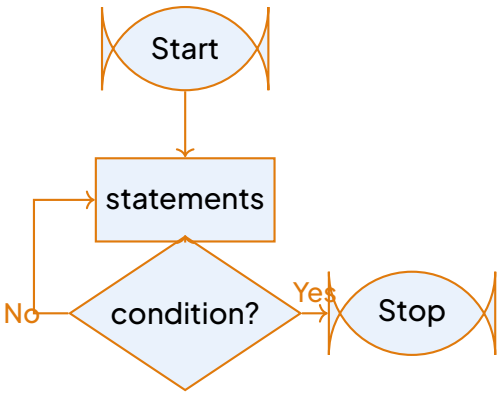
WHILE loop pattern: decision first, loop-back arrow from process to decision.

3.10 REPEAT...UNTIL loop (post-test)

Pseudocode:

```
REPEAT
\quad statements
UNTIL condition
```

Flowchart pattern — the process comes first, then the decision; the "No" branch loops back:



REPEAT…UNTIL pattern: process first, then decision; loop back on No (condition not yet true).

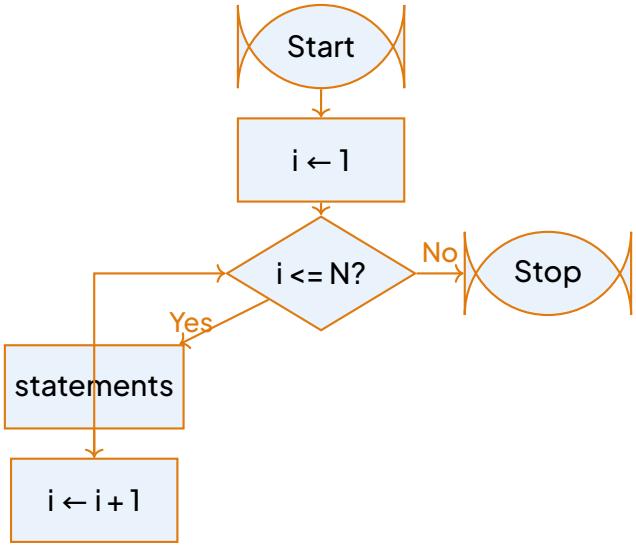
3.11 FOR loop (counted loop)

A `FOR` loop is a specialised pre-test loop. In a flowchart it is drawn as a `WHILE` with explicit initialisation and increment steps.

Pseudocode:

```
FOR i $\leftarrow$ 1 TO N DO
    \quad statements
        END FOR
```

Flowchart expansion:



FOR loop expanded into initialisation, decision, body, increment and loop-back.

WARNING **Loop-back arrow direction**

In a `WHILE` or `FOR` flowchart, the loop-back arrow goes from the process (or increment) back up to the decision diamond. In a `REPEAT` flowchart, the loop-back arrow goes from the "No" exit of the decision back up to the process. Drawing the arrow in the wrong direction is a frequent error that changes the logic entirely.

3.12 Complete worked flowcharts

We now draw full flowcharts for four classic problems. Study the layout, the labelling of branches, and the use of connectors where needed.

3.13 Example 1: Largest of three numbers

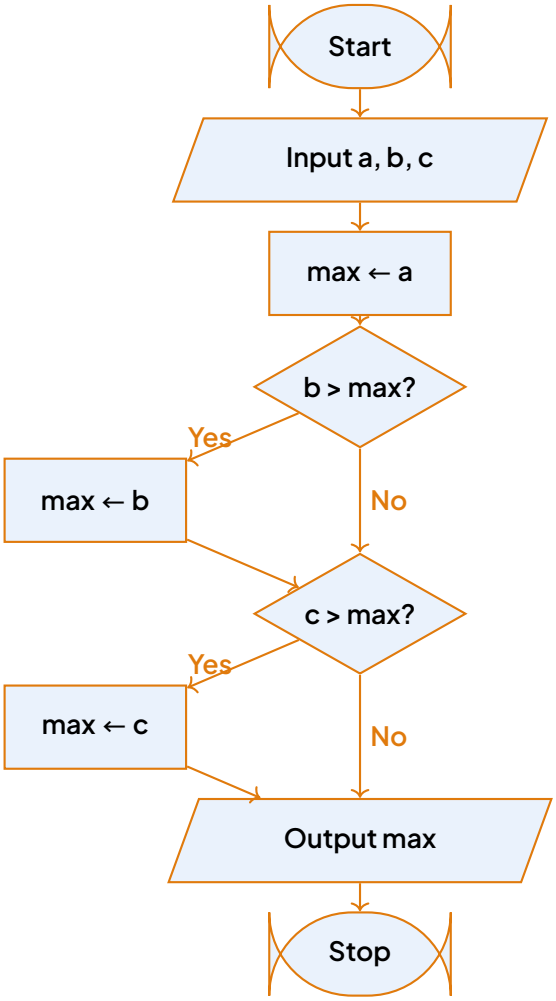
Problem: Input three numbers `a`, `b`, `c` and output the largest.

Pseudocode:

```
Input a, b, c
max $\leftarrow$ a
IF b > max THEN max $\leftarrow$ b END IF
IF c > max THEN max $\leftarrow$ c END IF
Output max
```

WORKED EXAMPLE 10 **Largest of three numbers — flowchart**

Flowchart:



Flowchart for finding the largest of three numbers.

SOLUTION

Explanation:

- The first decision tests `b > max`. If true, `max` becomes `b`; otherwise `max` stays unchanged. Both paths converge at the second decision.
- The second decision tests `c > max` (where `max` is now the larger of `a` and `b`). If true, `max` becomes `c`.
- Finally `max` is output. Every decision has two labelled branches; all paths reach Stop.

3.14 Example 2: Sum of N numbers

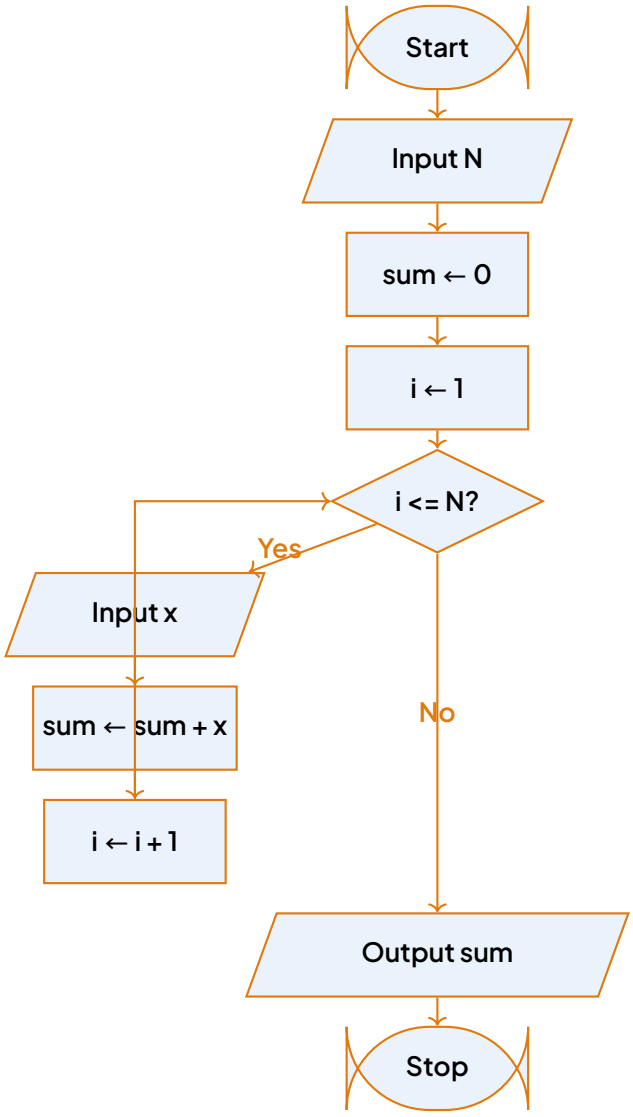
Problem: Input a positive integer `N`, then input `N` numbers and output their sum.

Pseudocode:

```
Input N
sum $\leftarrow$ 0
FOR i $\leftarrow$ 1 TO N DO
\quad Input x
\quad sum $\leftarrow$ sum + x
END FOR
Output sum
```

WORKED EXAMPLE 11 **Sum of N numbers — flowchart**

Flowchart:



Flowchart for summing N numbers using a FOR loop.

SOLUTION

Key points:

- Initialisation (`sum $\leftarrow$ 0, i $\leftarrow$ 1`) appears before the decision diamond.
- The loop body contains two process rectangles (input and addition) — each is a separate step.
- The increment `i $\leftarrow$ i + 1` is a process rectangle inside the loop, before the loop-back arrow.
- The "No" branch from the decision goes directly to output.

3.15 Example 3: Average of marks with sentinel −1 (WHILE pattern)

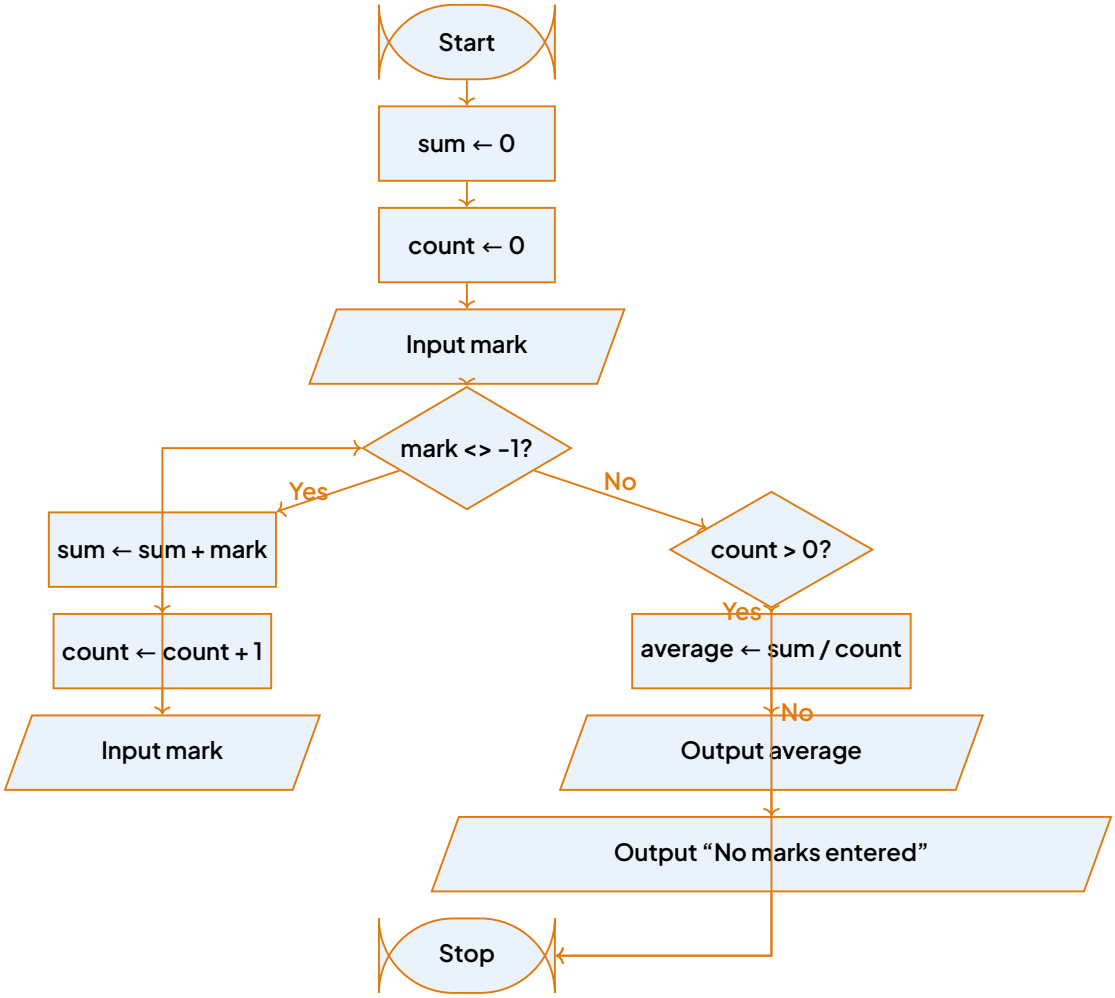
Problem: Input marks one by one until the user enters `-1` (sentinel). Compute and output the average of the valid marks (excluding the sentinel). If no valid marks are entered, output a message.

Pseudocode:

```
sum $\leftarrow$ 0
count $\leftarrow$ 0
Input mark
WHILE mark <> -1 DO
\quad sum $\leftarrow$ sum + mark
\quad count $\leftarrow$ count + 1
\quad Input mark
END WHILE
IF count > 0 THEN
\quad average $\leftarrow$ sum / count
\quad Output average
ELSE
\quad Output ``No marks entered''
END IF
```

WORKED EXAMPLE 12 **Average with sentinel −1 — complete flowchart**

Flowchart:



Complete flowchart for average of marks with sentinel -1 (WHILE pattern).

SOLUTION

Explanation of the WHILE pattern here:

- The first Input mark (priming read) is before the loop. This is essential: the condition `mark <> -1` must have a value to test.
- Inside the loop: update `sum` and `count`, then read the next mark (`Input mark`) before looping back to the decision.
- The loop-back arrow goes from the second `Input mark` up to the decision diamond.
- After the loop, an `IF...ELSE` handles the case `count = 0` (division by zero protection).
- Every decision has two labelled branches (Yes/No). All paths reach Stop.

3.16 Example 4: Grading with CASE (already shown above)

The grading flowchart in the previous subsection is a complete example of a CASE structure implemented as a chain of decision diamonds. Notice how each "Yes" branch goes to an action rectangle that then jumps to the common `Output grade` step, while each "No" branch proceeds to the next test.

3.17 Converting between pseudocode and flowchart

This is a core exam skill. You must be able to:

1. **Pseudocode → Flowchart:** Read the pseudocode line by line; for each construct (sequence, IF, WHILE, etc.) draw the corresponding pattern; connect them in order.
2. **Flowchart → Pseudocode:** Trace the flowchart from Start, writing the equivalent pseudocode statement for each symbol you encounter. Use indentation to show nesting.

METHOD **Pseudocode to Flowchart**

- Identify the main constructs in order: sequence, selections, loops.
- Draw the Start terminal.
- For each line of pseudocode:
  - Assignment / calculation → Process rectangle.
  - Input / Output → Parallelogram.
  - IF condition THEN … END IF → Decision diamond + one branch.
  - IF … ELSE … END IF → Decision diamond + two branches rejoining.
  - WHILE condition DO … END WHILE → Decision diamond before body, loop-back from end of body to diamond.
  - REPEAT … UNTIL condition → Body first, then decision diamond, loop-back on No.
  - FOR … TO … DO … END FOR → Initialisation, decision, body, increment, loop-back.
- Draw Stop terminal. Check that every path reaches Stop and every decision has two labelled exits.

METHOD **Flowchart to Pseudocode**

- Start at the Start terminal. Follow the arrows.
- When you hit a symbol, write the corresponding pseudocode line:

- Process rectangle → assignment or calculation.
- Parallelogram → Input or Output.
- Decision diamond → IF / WHILE / REPEAT depending on where the loop-back goes.

- Use indentation to show which statements are inside a selection or loop.
- Stop when you reach the Stop terminal.

3.18 Reading and interpreting a given flowchart (exam skill)

In the GCE exam you may be given a flowchart and asked:

- "State the purpose of this algorithm."
- "Complete the trace table for the inputs: 5, 12, -1."
- "What is the output if the input is ...?"

EXEMPLE **Interpreting a flowchart**

Consider the following flowchart (simplified description):

1. Start
2. Input `x`
3. `sum $\leftarrow$ 0`
4. `i $\leftarrow$ 1`
5. Decision: `i <= x`?
6. Yes: `sum $\leftarrow$ sum + i`; `i $\leftarrow$ i + 1`; loop back to decision
7. No: Output `sum`
8. Stop

Purpose: It computes the sum of integers from 1 to `x` (i.e., $1 + 2 + \cdots + x$). Trace for x = 4:

i	i <= x?	sum	Output
1	Yes	1	
2	Yes	3	
3	Yes	6	
4	Yes	10	
5	No	10	10

DID YOU KNOW? **Flowcharts in Cameroon's digital projects**

The ANTIC (Agence Nationale des Technologies de l'Information et de la Communication) and e-government projects (like the `e-SIAC` school management system) use flowcharts during the design phase to specify workflows for student registration, exam result processing and mobile money payment integration. A clear flowchart ensures that developers in Yaoundé, Douala and Buea all implement the same logic.

3.19 Worked examination-style exercise

WORKED EXAMPLE 13 **GCE-style: Draw a flowchart for the following problem**

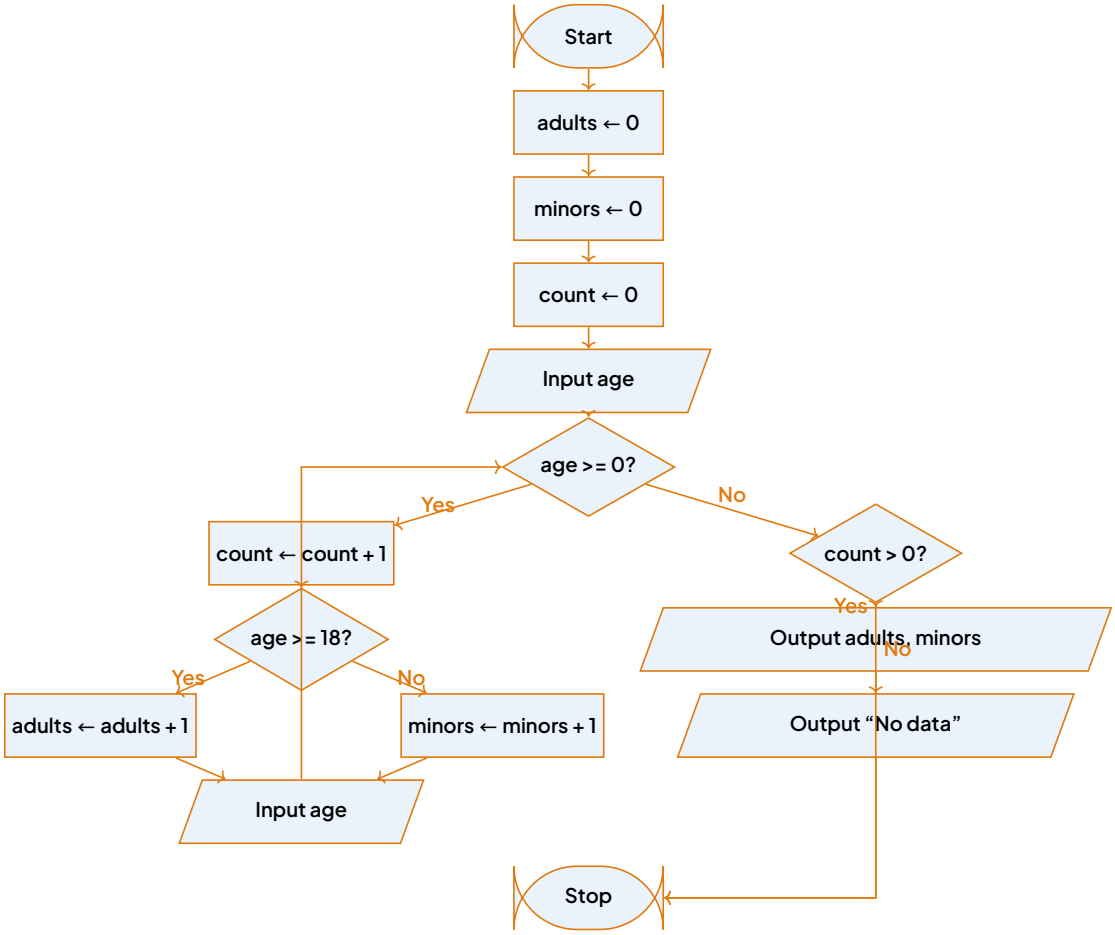
Problem: A program reads the ages of students in a class. The input ends when a negative age is entered. The program outputs the number of students who are 18 years or older (adults) and the number who are under 18 (minors). If no valid ages are entered, it outputs "No data".

Solution:

Step 1: Write pseudocode first (helps structure the flowchart).

```
adults $\leftarrow$ 0
minors $\leftarrow$ 0
count $\leftarrow$ 0
Input age
WHILE age >= 0 DO
\quad count $\leftarrow$ count + 1
\quad IF age >= 18 THEN
\quad\quad adults $\leftarrow$ adults + 1
\quad ELSE
\quad\quad minors $\leftarrow$ minors + 1
\quad END IF
\quad Input age
END WHILE
IF count > 0 THEN
\quad Output adults, minors
ELSE
\quad Output ``No data''
END IF
```

Step 2: Draw the flowchart.



Flowchart for counting adults and minors with sentinel-controlled input.

SOLUTION

Examiner's checklist (what earns marks):

- Correct symbols for Start, Stop, Input, Process, Decision.
- Priming read `Input age` before the WHILE decision.
- Decision `age >= 0?` with Yes/No labels; loop-back from second `Input age` to decision.
- Nested IF inside the loop: decision `age >= 18?` with two process rectangles (adults/minors) both leading to the next `Input age`.
- Post-loop IF `count > 0?` with two output branches.
- All paths reach Stop. No missing arrows. Neat top-to-bottom layout.

3.20 Practice exercises

EXERCISE 3 **Exercise 1: Flowchart to pseudocode**

The flowchart below describes an algorithm. Write the equivalent pseudocode.

1. Start
2. Input `n`
3. `fact $\leftarrow$ 1`
4. `i $\leftarrow$ 1`
5. Decision: `i <= n`?
6. Yes: `fact $\leftarrow$ fact * i`; `i $\leftarrow$ i + 1`; loop back to decision
7. No: Output `fact`
8. Stop

EXERCISE 4 **Exercise 2: Draw the flowchart**

Draw a complete flowchart for the following pseudocode. Use correct symbols, label all decision branches, and ensure every path reaches Stop.

```
Input x, y
IF x > y THEN
\quad max $\leftarrow$ x
\quad min $\leftarrow$ y
ELSE
\quad max $\leftarrow$ y
\quad min $\leftarrow$ x
END IF
Output max, min
```

EXERCISE 5 **Exercise 3: Sentinel-controlled loop with validation**

A program reads positive integers until the user enters 0. It counts how many of the entered numbers are divisible by 3. Draw the flowchart. (Hint: use a WHILE pattern with priming read; inside the loop, use an IF to test divisibility.)

SOLUTION **Exercise 1**

```
Input n
fact $\leftarrow$ 1
i $\leftarrow$ 1
WHILE i <= n DO
\quad fact $\leftarrow$ fact * i
\quad i $\leftarrow$ i + 1
END WHILE
Output fact
```

This computes `n!` (factorial of n).

SOLUTION **Exercise 2**

Flowchart:

- Start → Input x, y (parallelogram)
- Decision: `x > y?` (diamond)
- Yes branch: Process `max $\leftarrow$ x`; Process `min $\leftarrow$ y`
- No branch: Process `max $\leftarrow$ y`; Process `min $\leftarrow$ x`
- Both branches rejoin → Output max, min (parallelogram) → Stop

SOLUTION **Exercise 3**

Key points for the flowchart:

- Start; `count $\leftarrow$ 0`; Input `num` (priming read).
- Decision: `num <> 0?` (or `num != 0`).
- Yes: Decision `num MOD 3 = 0?` → Yes: `count $\leftarrow$ count + 1`; No: (nothing).
- Both branches of inner IF go to `Input num` (next read) → loop back to outer decision.
- Outer decision No branch → Output `count` → Stop.

KEY POINTS **Exam checklist for flowcharts**

- Symbols: Oval for Start/Stop, Parallelogram for Input/Output, Rectangle for Process, Diamond for Decision, Circle for Connector.
- Arrows: On every flow line, pointing in the correct direction.
- Decisions: Exactly two exits, labelled Yes/No (or True/False).
- Loops: WHILE/FOR — decision before body, loop-back to decision. REPEAT — body before decision, loop-back on No.
- Priming read: For sentinel-controlled WHILE loops, the first Input must be before the loop.
- Completeness: Every path reaches Stop. No dangling arrows.
- Neatness: Top-to-bottom flow, adequate spacing, readable text.

WARNING **Last-minute traps**

- Drawing a rectangle for `IF condition` — it must be a diamond.
- Forgetting the loop-back arrow in a WHILE or FOR loop.
- In a REPEAT loop, looping back on Yes instead of No.
- Missing the priming read before a sentinel WHILE loop (causes infinite loop or missed first value).
- Not labelling the two branches of a decision — the examiner cannot guess which is which.
- Using a connector circle incorrectly (e.g., only one A, or A connected to B).

You now have the complete toolkit to draw, read and convert flowcharts for any algorithm built from sequence, selection and iteration. In the next section we will practise **dry running** (trace tables) — the systematic way to verify that your flowchart or pseudocode produces the correct results for given test data.

4 DRY RUNNING ALGORITHMS (TRACE TABLES)

In the previous section we learned how to draw flowcharts that describe the logic of an algorithm visually. A flowchart, however, only shows what the algorithm is supposed to do. How can we be sure that it actually does it, before spending time translating it into a programming language? Imagine a carpenter in Buea who builds a chair: before delivering it, he sits on it to check that it does not collapse. The programmer's equivalent of "sitting on the chair" is the **dry run**: we execute the algorithm by hand, with pen and paper, using carefully chosen test data, and we

watch the values of the variables evolve step by step. This simple technique, also called **desk checking** or **tracing**, is one of the most powerful verification tools a programmer possesses — and one of the most frequently examined skills at the GCE Advanced Level.

4.1 What is dry running and why do we do it?

DEFINITION **Dry running**

Dry running (also called desk checking or tracing) is the manual, step-by-step execution of an algorithm with chosen test data, in which the values of all variables and all outputs are recorded at each step in a table called a **trace table**.

The purposes of a dry run are the following:

- **Verify the logic:** does the algorithm produce the expected result for the chosen input?
- **Predict the output:** given the algorithm and the input, determine exactly what will be displayed — a classic examination question.
- **Find errors before coding:** it is far cheaper to correct a mistake on paper than after hours of programming. Errors found at the design stage cost minutes; errors found after coding can cost days.
- **Understand an unfamiliar algorithm:** tracing an algorithm written by someone else is the best way to understand how it works.

DID YOU KNOW? **Why "dry" running?**

The word "dry" means that the algorithm is executed without a computer — the machine stays "dry", unused. Professional software teams still practise desk checking today: before a mobile money application is deployed by an operator in Cameroon, testers walk through the transaction algorithms by hand with sample amounts to make sure no customer can be debited twice or credited by mistake.

4.2 The trace table

The trace table is the tool that makes a dry run rigorous instead of chaotic.

DEFINITION **Trace table**

A **trace table** is a table with one column per variable (and one column per output), and one row per executed step or iteration. Each row records the values of the variables after the corresponding instruction has been executed.

METHOD **Constructing a trace table**

1. Read the algorithm carefully and list all its variables; create one column for each variable, plus a column for the output if the algorithm displays anything.
2. Write the initial values of the variables in the first row (before or just after initialisation).
3. Execute the algorithm instruction by instruction, exactly as the computer would — never skip a step and never assume.
4. After each instruction that changes a value, write a new row showing the updated values; leave unchanged cells blank or repeat the value (but be consistent).
5. For a loop, record the condition test as well as the body: one row per iteration, including the final test that fails.
6. Record every output in the output column at the moment it occurs.
7. Compare the final output with the expected result: if they differ, the algorithm contains an error.

WARNING **The computer never cheats**

When dry running, you must behave like a machine: execute exactly what is written, not what you think was intended. Most tracing errors made by students come from "correcting" the algorithm in their head while tracing it. If the algorithm says `fact $\leftarrow$ fact + i`, then you add, even if you believe multiplication was intended — and your trace will reveal the bug precisely because you were faithful.

4.3 Dry running a selection algorithm

With a selection structure, the key question is: which branch is taken? A complete dry run therefore requires **several traces**, one for each possible path through the algorithm.

Consider the algorithm that finds the largest of three numbers a, b, c:

```
1  read a, b, c
2  max ← a
3  if b > max then max ← b endif
4  if c > max then max ← c endif
5  write max
```

Trace 1: input $a = 7$, $b = 3$, $c = 5$ (largest first).

Step	a	b	c	max	Comment
1	7	3	5	–	read inputs
2	7	3	5	7	$max \leftarrow a$
3	7	3	5	7	3 > 7? false, branch skipped
4	7	3	5	7	5 > 7? false, branch skipped
5	7	3	5	7	output: 7 ✓

Trace 2: input $a = 2$, $b = 9$, $c = 4$ (largest in the middle). At step 3, $9 > 2$ is true, so max becomes 9; at step 4, $4 > 9$ is false. Output: 9 ✓.

Trace 3: input $a = 1$, $b = 4$, $c = 8$ (largest last). At step 3, max becomes 4; at step 4, $8 > 4$ is true, so max becomes 8. Output: 8 ✓.

The three orderings exercise the different combinations of branches. Only after all three traces agree with the expected answers can we be reasonably confident the algorithm is correct.

METHOD **Choosing test data**

Good test data must exercise every branch and every boundary of the algorithm:

1. Normal values: typical inputs in the middle of the expected range (e.g. a mark of 12/20).
2. Boundary (extreme) values: values exactly at the limits of a condition (e.g. the pass mark 10/20 itself, or 0 and 20). Boundary values reveal "off-by-one" errors such as writing > instead of ≥.
3. Invalid values: inputs outside the allowed range (e.g. a mark of -3 or 25) to check that the algorithm rejects them properly.
4. For loops: data that produce the first, middle and last iterations, and if possible zero iterations (a loop whose condition is false from the start).

4.4 Dry running a loop: the factorial algorithm

Loops are where dry running is most valuable, because a small error is repeated at every iteration. Let us trace the algorithm that computes the factorial of a positive integer n, that is $n! = 1 \times 2 \times \cdots \times n$:

```
1  read n
2  fact ← 1
3  i ← 1
4  while i ≤ n do
5      fact ← fact × i
6      i ← i + 1
7  endwhile
8  write fact
```

EXEMPLE **Complete dry run of the factorial algorithm for $n = 4$**

We expect $4! = 24$. The trace table has one column per variable (n, $fact$, i), one column for the loop test, and one row per step:

Step	n	$fact$	i	Test $i \le n$	Comment
1	4	–	–	–	read n
2	4	1	–	–	initialise $fact$
3	4	1	1	–	initialise counter i
4	4	1	1	$1 \le 4$: true	enter loop (iteration 1)
5–6	4	1	2	–	$fact = 1 \times 1$; i becomes 2
4	4	1	2	$2 \le 4$: true	iteration 2
5–6	4	2	3	–	$fact = 1 \times 2$; i becomes 3
4	4	2	3	$3 \le 4$: true	iteration 3
5–6	4	6	4	–	$fact = 2 \times 3$; i becomes 4
4	4	6	4	$4 \le 4$: true	iteration 4
5–6	4	24	5	–	$fact = 6 \times 4$; i becomes 5
4	4	24	5	$5 \le 4$: false	exit loop
8	4	24	5	–	output: 24 ✓

The loop body is executed exactly 4 times ($i = 1, 2, 3, 4$), the condition is tested 5 times, and the final output 24 matches the expected value of $4!$.

Trace table: factorial of 4

Step	n	$fact$	i	Test
init	4	1	1	–
test 1	4	1	1	true
iter 1	4	1	2	–
iter 2	4	2	3	–
iter 3	4	6	4	–
iter 4	4	24	5	–
test 5	4	24	5	**false**

loop exits here

Output: $fact = 24 = 4!$

Completed trace table for the factorial algorithm with $n = 4$. Note the final failing test (highlighted): the condition is checked one last time and the body is not executed again.

WARNING **The WHILE condition is tested before each iteration**

A very common mistake is to forget that a `WHILE` loop tests its condition before every iteration — including the last one. In the factorial trace, the condition $i \le n$ is tested five times for only four executions of the body: the fifth test ($5 \le 4$) fails and the body is skipped. Students who stop their trace at iteration 4, or who execute the body once more "because i was 4", lose marks and misunderstand the loop. Always write the row of the final failing test.

4.5 Dry running a sentinel-controlled loop

A **sentinel** is a special value that signals the end of input. The following algorithm reads examination marks one by one and computes their average; the sentinel value -1 means "no more marks":

```
1  sum ← 0; count ← 0
2  read mark
3  while mark ≠ -1 do
4      sum ← sum + mark
5      count ← count + 1
6      read mark
7  endwhile
8  if count > 0 then avg ← sum/count; write avg
9  else write "no marks entered" endif
```

WORKED EXAMPLE 14 **Dry run of the sentinel-controlled average**

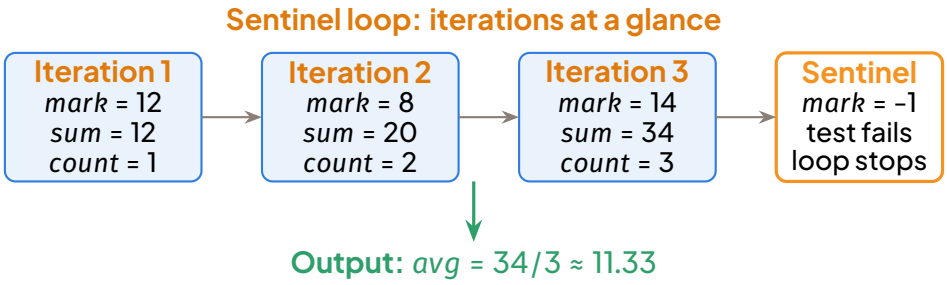
Trace the algorithm above with the input sequence 12, 8, 14, -1 and state the output.

SOLUTION

We expect the average $(12 + 8 + 14)/3 = 34/3 \approx 11.33$.

Step	*mark*	*sum*	*count*	Test *mark* ≠ -1	Comment
1	–	0	0	–	initialisation
2	12	0	0	–	first read
3	12	0	0	true	iteration 1
4–6	8	12	1	–	add 12; read next
3	8	12	1	true	iteration 2
4–6	14	20	2	–	add 8; read next
3	14	20	2	true	iteration 3
4–6	-1	34	3	–	add 14; read sentinel
3	-1	34	3	false	exit loop
8	-1	34	3	–	*avg* = 34/3 ≈ 11.33

Output: 11.33 (approximately). The trace confirms that the sentinel -1 is not added to the sum, because the test at step 3 rejects it before the body runs. Note also the role of step 8: if the very first value read were -1, the loop would execute zero times, *count* would remain 0, and the `else` branch would prevent a division by zero. A dry run with input -1 alone verifies this boundary case.



Summary of the trace of the sentinel-controlled average: three iterations accumulate the marks; the sentinel -1 makes the test fail and stops the loop without being processed.

4.6 Detecting and correcting errors from a trace

The real power of dry running appears when the trace disagrees with the expected result: the

algorithm contains a design error, and the trace tells us exactly where. Here are the four classic errors a trace reveals:

- **Wrong initialisation.** If the factorial algorithm began with $fact \leftarrow 0$, every row of the trace would show $fact = 0$ (since $0 \times i = 0$). A column that never changes, or changes wrongly from the first row, signals a bad initial value.
- **Off-by-one bounds.** Writing `while` $i < n$ instead of $i \le n$ makes the loop stop one iteration too early: for $n = 4$ the trace would end with $fact = 6$ (which is 3!) instead of 24. Comparing the number of iterations with the expected number exposes this immediately.
- **Infinite loops.** If the instruction $i \leftarrow i + 1$ is forgotten, the trace shows $i = 1$ forever and the test $i \le n$ is true at every row: the loop never ends. A trace in which the condition column never changes is the signature of an infinite loop.
- **Wrong branch condition.** If the "largest of three" algorithm used $max \leftarrow b$ whenever $b < max$, the traces with different input orderings would produce contradictory answers, revealing the inverted comparison.

EXEMPLE **Correcting a faulty algorithm at the design stage**

A student writes the following algorithm to compute the sum $1 + 2 + \cdots + n$:

```
1  read n
2  sum ← 0; i ← 0
3  while i < n do
4      i ← i + 1
5      sum ← sum + i
6  endwhile
7  write sum
```

Dry run with $n = 3$ (expected result: $1 + 2 + 3 = 6$):

Step	i	sum	Test $i < n$	Comment
2	0	0	–	initialisation
3	0	0	true	iteration 1
4–5	1	1	–	add 1
3	1	1	true	iteration 2
4–5	2	3	–	add 2
3	2	3	true	iteration 3
4–5	3	6	–	add 3
3	3	6	false	exit

The output 6 is correct for $n = 3$ — but a single successful trace is not a proof. Testing the boundary case $n = 1$: the test $0 < 1$ is true, i becomes 1, sum becomes 1, then $1 < 1$

fails; output 1, correct. Testing $n = 0$ (no terms to add, expected sum 0): the test $0 < 0$ fails immediately and the output is 0, also correct. The algorithm survives these traces. Suppose instead the student had written `while` $i \leq n$: the trace for $n = 3$ would give $sum = 10$ (it would add 4 as well), and the off-by-one error would be caught and corrected by changing the condition to $i < n$ before any code is written. This is debugging at the design stage.

WARNING **One successful trace is never enough**

An algorithm can give the right answer for one input and be wrong for another (especially boundary values). Always dry run with several data sets chosen to exercise every branch and both ends of every loop. Conversely, one failing trace is enough to prove the algorithm is wrong.

KEY POINTS **Key points**

- Dry running is the manual execution of an algorithm with test data; results are recorded in a trace table: one column per variable and per output, one row per step.
- Trace every instruction faithfully, including the final failing test of a WHILE loop.
- Choose test data covering normal, boundary and invalid values, and every branch of a selection.
- A trace reveals wrong initialisation, off-by-one bounds, infinite loops and wrong conditions — and lets you correct them before coding.

DID YOU KNOW? **GCE exam tip**

Trace-table questions appear very frequently at the GCE Advanced Level, either as "complete the trace table" or "state the output of this algorithm". To earn full marks: write every row, including initialisation and the final failing test of each loop; keep the columns aligned with the variable names; and state the final output clearly. A trace with missing rows, even if the final answer is right, usually loses method marks.

EXERCISE 6 **Trace and debug**

The following algorithm is supposed to display the squares of the integers from 1 to 5 (i.e.

1, 4, 9, 16, 25):

```
1   i ← 1
2   while i < 5 do
3       write i × i
4       i ← i + 1
5   endwhile
```

1. Construct the complete trace table for this algorithm.
2. State the actual output and explain why it is wrong.
3. Correct the algorithm and verify your correction with a new dry run.

SOLUTION **Exercise 1**

1. Trace table:

Step	i	Test $i < 5$	Output
1	1	–	–
2	1	true	–
3–4	2	–	1
2	2	true	–
3–4	3	–	4
2	3	true	–
3–4	4	–	9
2	4	true	–
3–4	5	–	16
2	5	false	–

2. The actual output is 1, 4, 9, 16: the value 25 is missing. The condition $i < 5$ fails when $i = 5$, so the body never runs for $i = 5$ — an off-by-one error.
3. Correction: replace the condition by $i \leq 5$. A new dry run then shows a fifth iteration: test $5 \leq 5$ true, output 25, i becomes 6, test $6 \leq 5$ false, loop exits. Output: 1, 4, 9, 16, 25, as required.

Dry running closes the design phase of problem solving: once the flowchart is drawn and the trace tables confirm that the logic is correct for normal, boundary and invalid data, we can move

to the next stage — translating the verified algorithm into a real programming language, which is the subject of the following section on writing code.

5 WRITING CODE AND INTEGRATION

In the previous section we learnt how to dry run an algorithm: we executed our pseudocode "by hand" with a trace table and predicted exactly what values each variable would hold. That skill now becomes our safety net. In this final section we complete the journey from a problem statement to a working program: we **translate** the design into a real programming language, **run** it, and **test** it against the predictions of our trace table. This is where algorithms stop being paper exercises and become tools that a computer can execute — the very heart of the GCE practical and theory papers.

5.1 From design to code: the idea of implementation

DEFINITION **Implementation (coding)**

Implementation (or **coding**) is the translation of an algorithm — expressed in pseudocode or as a flowchart — into a **programming language** so that a computer can execute it. The resulting text is called the **source code** (or program).

The key observation is that the thinking has already been done during design. Coding is a **translation exercise**: every pseudocode construct has an exact equivalent in a high-level language. Throughout this course we use **Python** as the reference language, because its syntax is close to pseudocode and it is the language most often used in Cameroonian schools and in GCE-style questions.

Construct	Pseudocode	Python
Output	DISPLAY "Hello"	`print("Hello")`
Input	READ mark	`mark = float(input("Mark? "))`
Assignment	total ← total + mark	`total = total + mark`
Selection	IF mark ≥ 10 THEN … ELSE … ENDIF	`if mark >= 10: … else: …`
Pre-test loop	WHILE n > 0 DO … ENDWHILE	`while n > 0: …`
Counted loop	FOR i ← 1 TO 10 DO … ENDFOR	`for i in range(1, 11): …`

WARNING **Python syntax traps**

In Python the colon `:` after `if`, `elif`, `else`, `while` and `for` is compulsory, and the body of each block is shown by indentation (usually 4 spaces), not by ENDIF or ENDWHILE. A missing colon or inconsistent indentation is a syntax error. Also note: `=` assigns a value, `==` tests equality. Writing `if mark = 10:` is an error; in C-like languages it is even worse because it

compiles and silently gives wrong results.

Pseudocode	Python

IF age ≥ 18 THEN
  DISPLAY “Adult”
ELSE
  DISPLAY “Minor”
ENDIF
	`if age >= 18:`
	`  print("Adult")`
	`else:`
	`  print("Minor")`

n ← 5
WHILE n > 0 DO
  DISPLAY n
  n ← n - 1
ENDWHILE
	`n = 5`
	`while n > 0:`
	`  print(n)`
	`  n = n - 1`

Side-by-side translation: an IF…ELSE selection and a WHILE loop, from pseudocode to Python.

5.2 Writing code 1 — selection programs

Selection programs translate the IF…THEN…ELSE and CASE structures studied earlier. Let us code the classics.

Example 1: grading a mark. A mark out of 20 is graded: “Fail” below 10, “Pass” from 10 to 13.99, “Credit” from 14 to 15.99, “Distinction” from 16 upwards.

EXEMPLE **Grading a mark in Python**

```
mark = float(input("Enter the mark out of 20: "))
if mark >= 16:
\quad grade = "Distinction"
elif mark >= 14:
\quad grade = "Credit"
elif mark >= 10:
\quad grade = "Pass"
else:
\quad grade = "Fail"
print("Grade:", grade)
```

The tests are checked in order: as soon as one is true, its block runs and the rest are skipped. That is why we can write `mark >= 14` without also writing `mark < 16`.

Example 2: largest of three numbers. Read *a*, *b*, *c* and display the largest.

EXEMPLE **Largest of three numbers**

```
a = float(input("First number: "))
b = float(input("Second number: "))
c = float(input("Third number: "))
largest = a
if b > largest:
\quad largest = b
if c > largest:
\quad largest = c
print("The largest is", largest)
```

The strategy is to keep a "champion" variable and let each challenger try to beat it — exactly the algorithm we dry ran in the previous section.

Example 3: leap year. A year is a leap year if it is divisible by 4, except centuries, which must be divisible by 400.

EXEMPLE **Leap year test**

```
year = int(input("Enter a year: "))
if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
\quad print(year, "is a leap year")
else:
\quad print(year, "is not a leap year")
```

Here % is the **modulo** operator (remainder of integer division). Year 2000 is a leap year (divisible by 400); year 1900 is not (divisible by 100 but not by 400); year 2024 is (divisible by 4, not by 100).

Example 4: a simple menu. Menus translate the CASE structure: a mobile-money style menu with several branches.

EXEMPLE **Menu with multiple branches**

```
print("1. Check balance")
print("2. Send money")
print("3. Buy airtime")
choice = input("Your choice: ")
if choice == "1":
\quad print("Your balance is displayed.")
```

```
elif choice == "2":
\quad print("Enter the receiver's number.")
elif choice == "3":
\quad print("Enter the amount of airtime.")
else:
\quad print("Invalid choice.")
```

5.3 Writing code 2 — loop programs

Loops translate the WHILE and FOR structures. The discipline is always the same: **initialise** before the loop, **update** inside the loop, and check the **termination condition**.

Example 1: multiplication table. Display the table of 7, from 7×1 to 7×12.

EXEMPLE **Multiplication table with a for loop**

```
n = int(input("Which table? "))
for i in range(1, 13):
\quad print(n, "x", i, "=", n * i)
```

`range(1, 13)` generates $1, 2, \ldots, 12$ — remember the upper bound is excluded, a classic source of off-by-one errors.

Example 2: sum and average of N numbers.

EXEMPLE **Sum and average**

```
n = int(input("How many numbers? "))
total = 0
for i in range(1, n + 1):
\quad x = float(input("Enter number: "))
\quad total = total + x
average = total / n
print("Sum =", total, " Average =", average)
```

The variable `total` is an **accumulator**: it must be initialised to `0` before the loop. Forgetting this is one of the most frequent logic errors.

Example 3: factorial. $n! = 1 \times 2 \times \cdots \times n$.

EXEMPLE **Factorial**

```
n = int(input("Enter n: "))
fact = 1
for i in range(2, n + 1):
\quad fact = fact * i
print(n, "! =", fact)
```

Note the accumulator starts at `1` (not `0`!) because we multiply. For $n = 5$ the loop computes $1 \times 2 \times 3 \times 4 \times 5 = 120$.

Example 4: sentinel-controlled input validation. We keep asking until the user enters a valid mark between 0 and 20.

EXEMPLE **Input validation with a while loop**

```
mark = float(input("Enter a mark (0-20): "))
while mark < 0 or mark > 20:
\quad print("Invalid! A mark must be between 0 and 20.")
\quad mark = float(input("Enter a mark (0-20): "))
print("Mark accepted:", mark)
```

This is a **sentinel-controlled** loop: the loop continues while the value is "bad" and stops when a valid value (the sentinel condition) is met. The first input before the loop is essential — without it, `mark` would be undefined when the test is first evaluated.

5.4 Writing code 3 — combined programs

Real problems combine sequence, selection and loops. This is exactly what the GCE integration questions expect.

Example 1: processing a class list. Enter the marks of a class, count passes and failures (pass mark 10/20), and find the best mark.

WORKED EXAMPLE 15 **Class list processing**

Statement: Write a Python program that reads the number of students, then each mark out of 20, and displays the number of passes, the number of failures and the best mark.

SOLUTION

Design first (pseudocode sketch): initialise passes ← 0, fails ← 0, best ← -1; loop over

the students, read each mark, update the counters with an IF, and update best if the mark beats it; display the three results.

```
n = int(input("Number of students: "))
passes = 0
fails = 0
best = -1
for i in range(1, n + 1):
\quad mark = float(input("Mark of student: "))
\quad if mark >= 10:
\quad\quad passes = passes + 1
\quad else:
\quad\quad fails = fails + 1
\quad if mark > best:
\quad\quad best = mark
print("Passes:", passes)
print("Failures:", fails)
print("Best mark:", best)
```

Why `best = -1`? Since every valid mark is at least 0, starting the champion at -1 guarantees the first real mark beats it. Test: with marks 12, 7, 15 the program outputs Passes: 2, Failures: 1, Best mark: 15 — which a quick dry run on paper confirms.

Example 2: a simple fees-checking program. A school asks whether each student has paid fees; the program keeps asking names until the clerk types "stop".

EXEMPLE **Fees check with a sentinel**

```
name = input("Student name (or 'stop'): ")
while name != "stop":
\quad paid = input("Has " + name + " paid? (yes/no): ")
\quad if paid == "yes":
\quad\quad print(name, "may sit the exam.")
\quad else:
\quad\quad print(name, "must see the bursar.")
\quad name = input("Student name (or 'stop'): ")
print("End of list.")
```

The sentinel value `"stop"` ends the loop. Notice the input appears twice: once before the loop (priming read) and once at the end of the body.

Example 3: number-guessing game. The computer chooses a secret number; the player guesses until correct, with "too big" / "too small" hints.

EXEMPLE **Guessing game**

```
secret = 42
guess = int(input("Guess the number: "))
while guess != secret:
\quad if guess > secret:
\quad\quad print("Too big!")
\quad else:
\quad\quad print("Too small!")
\quad guess = int(input("Try again: "))
print("Correct! The number was", secret)
```

This combines a WHILE loop (repeat until found) with an IF…ELSE (give the hint) — a perfect miniature of the integration activity.

5.5 Running and testing code: the three kinds of error

Once the code is written, we run it and compare its behaviour with our trace-table predictions. When something goes wrong, the error belongs to one of three families.

DEFINITION **Syntax, runtime and logic errors**

A **syntax error** is a violation of the grammar of the language (missing colon, unclosed parenthesis, bad indentation). It is detected by the interpreter or compiler before or during translation, and the program does not run. A **runtime error** appears while the program executes (division by zero, converting the text "abc" to a number) and usually crashes the program. A **logic error** is a mistake in the algorithm itself: the program runs to the end but produces wrong results (wrong initialisation, wrong comparison, off-by-one loop bound).

WARNING **The most dangerous error is the silent one**

Do not confuse syntax errors with logic errors. A syntax error is caught for you by the interpreter, with a line number. A logic error is invisible: the program runs happily and gives a wrong answer — for example computing an average as `total / (n + 1)`. This is precisely why the dry run of the previous section matters: the trace table gives you the expected values, so any deviation during testing reveals a logic error immediately.

METHOD **Testing a program against the dry run**

1. Before coding, dry run the algorithm with 2 or 3 well-chosen data sets (normal case, boundary case such as mark = 10, extreme case such as $n = 0$).
2. Write the code, then run it with exactly the same data sets.
3. Compare each output with the trace-table prediction, line by line.
4. If they differ, locate the first line where the program's variable values depart from the table — the bug is there or just before.
5. Correct, then re-run all the test sets (a fix can break something else).

5.6 Good programming practice

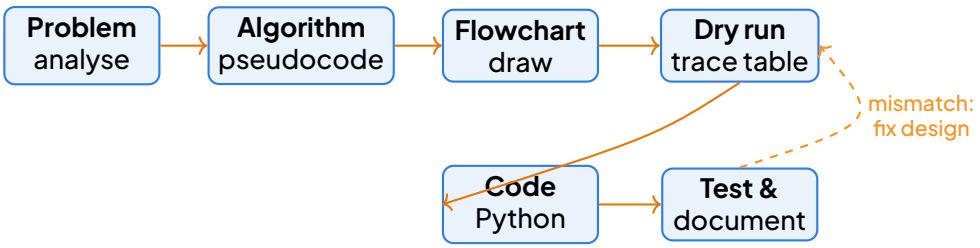
A program is read far more often than it is written — by you next month, by a classmate, by the examiner. Professional habits cost nothing and earn marks.

KEY POINTS **Good practice checklist**

- Meaningful identifiers: `total`, `best_mark`, `number_of_students` rather than `x`, `t`, `z`.
- Comments: short notes beginning with `#` in Python, explaining why, not what (`# champion strategy: keep the best mark so far`).
- Indentation: consistent 4-space indentation; in Python it is part of the syntax, in other languages it is a courtesy to the reader.
- Clear prompts and outputs: `input("Enter a mark (0-20): ")` tells the user what to type; `print("Average =", average)` labels the result instead of printing a bare number.
- One task per block: initialisation, input, processing, output — in that order.

5.7 Integration activity methodology

The integration activity is the GCE's way of checking that you can manage the whole development cycle, not just one step of it. The pipeline below summarises the method you must be able to describe and apply.



The development pipeline: analyse the problem, design (pseudocode and flowchart), dry run, code, then test against the trace-table predictions and document.

METHOD **Integration activity — the six steps**

1. Analyse the problem: identify the inputs, the outputs, and the processing; note special cases (empty list, invalid input).
2. Design the algorithm: write the pseudocode and draw the flowchart; the two must say the same thing.
3. Dry run: build a trace table with representative data and record the expected outputs.
4. Code: translate construct by construct into Python.
5. Test: run with the trace-table data; classify any error (syntax, runtime, logic) and correct it.
6. Document: add comments, a heading line (author, date, purpose), and keep the test results.

EXERCISE 7 **Integration: school canteen**

A canteen sells meals at 500 FCFA each. Write a complete solution (pseudocode, flowchart, trace table for the data 3, 0, 2, then Python code) for a program that reads the number of meals bought by each of 5 pupils and displays the total money collected and the number of pupils who bought nothing.

SOLUTION **Integration: school canteen**

Pseudocode: total ← 0; zeros ← 0; FOR i ← 1 TO 5: READ m; total ← total + m; IF m = 0 THEN zeros ← zeros + 1; ENDFOR; DISPLAY total × 500, zeros. Trace table for 3, 0, 2 (with 3 pupils for brevity): after the three iterations, total = 5, zeros = 1; expected output: 2500

FCFA and 1 pupil. Python:

```
total = 0
zeros = 0
for i in range(1, 6):
\quad m = int(input("Meals bought by pupil: "))
\quad total = total + m
\quad if m == 0:
\quad\quad zeros = zeros + 1
print("Money collected:", total * 500, "FCFA")
print("Pupils who bought nothing:", zeros)
```

Running it with 3, 0, 2 (and any two further values) confirms the trace-table predictions.

5.8 Beyond Python: recognising constructs in other languages (exam-useful extra)

GCE questions sometimes show a fragment in another language, typically C, and ask you to state its output or correct it. The logic is identical; only the clothing changes.

Construct	Python	C
Selection	`if x > 0: … else: …`	`if (x > 0) { ... } else { ... }`
Multi-branch	`if / elif / else`	`switch (x) { case 1: ...; break; ... }`
Pre-test loop	`while x > 0:`	`while (x > 0) { ... }`
Counted loop	`for i in range(1, 11):`	`for (i = 1; i <= 10; i++) { ... }`

In C, blocks are delimited by braces `{ }` instead of indentation, statements end with a semi-colon, and the counted loop packs initialisation, condition and update into the `for` header. Beware the classic C trap: `if (x = 5)` assigns 5 to *x* and is always true — the comparison must be `==`.

DID YOU KNOW? **Exam tip — reading code is as important as writing it**

In the GCE Advanced Level paper you may be asked to complete a short program with missing lines, to correct a buggy one, or to state the exact output of a given fragment. Train yourself to read code with a trace table in hand: write down the variables, follow each line, and predict the output before looking at the answer choices. Ten minutes of dry-running practice is worth more than an hour of memorising syntax.

KEY POINTS **Key points of the section**

- Implementation is the translation of a validated algorithm into a programming language; design and dry run always come first.
- Every pseudocode construct (IF, CASE, WHILE, FOR) has a direct Python equivalent: `if/elif/else`, `while`, `for … in range(...)`.
- Syntax errors are caught by the interpreter; runtime errors crash the program; logic errors run silently and are detected by testing against trace-table predictions.
- Good practice — meaningful names, comments, indentation, clear prompts — makes programs readable and earns marks.
- The integration activity follows six steps: analyse, design, dry run, code, test, document.

6 INTEGRATION ACTIVITY

6.1 Aims of the activity

This integration activity closes the chapter by asking you to use, in one single project, everything you have learnt: problem analysis, **selection** structures (`IF` and `CASE`), **loop** structures with a sentinel value, **counters** and **accumulators**, **flowchart** drawing, **dry running** with a trace table, and finally **coding** and **testing**. By the end of the activity you should be able to:

- analyse an authentic school problem and identify inputs, processes and outputs;
- design an algorithm in pseudocode and translate it into a complete flowchart;
- predict the behaviour of the algorithm by dry running it on a small data set;
- write a working program (Python is used here, but Pascal or C give the same logic);
- compare dry-run predictions with actual output and write a short test report.

KEY POINTS **Skills mobilised**

Analysis → pseudocode → flowchart → trace table (dry run) → code → testing. This is exactly the development cycle expected at the GCE Advanced Level practical and theory papers.

6.2 Situation

End-of-term at Government Bilingual High School Mendong.

It is the end of the third term. Mr. Ndi, the class master of Upper Sixth Science A, must hand two reports to the Principal before the general staff meeting: a fees report for the bursar and a results summary for the academic board. Doing this by hand for 45 students is slow and error-prone, so he asks your ICT group to write a small program.

The full school fees for the year amount to **50 000 FCFA** per student (PTA levy, examination registration contribution and sports fee included). For each student, Mr. Ndi has two pieces of information: the total fees paid so far (in FCFA) and the end-of-year average mark (out of 20).

The program must work as follows:

1. Repeatedly ask for a student's **name**, **fees paid** and **average mark**. The entry of students stops when the name `STOP` is typed (sentinel value).
2. For each student, determine the **fee status**:
   - `PAID UP` if fees paid = 50 000;
   - `PART PAYMENT` if fees paid is at least 25 000 but less than 50 000;
   - `DEFAULTER` if fees paid is less than 25 000.
3. For each student, determine the **grade** from the average mark m:

Average mark	Grade
$16 \le m \le 20$	A (Excellent)
$14 \le m < 16$	B (Very good)
$12 \le m < 14$	C (Good)
$10 \le m < 12$	D (Pass)
$m < 10$	F (Fail)

4. While reading the students, accumulate: the **number of students** processed, the **number of defaulters**, the **total fees collected**, the **sum of averages** (to compute the class average) and the **best mark** (with the name of the best student).
5. At the end, display a **summary report**: number of students, number of defaulters, total fees collected, class average (2 decimal places), best mark and best student's name.

Test data (to be used for the dry run and the first test of the program):

Name	Fees paid (FCFA)	Average (/20)
Amina	50000	15.5
Bih	20000	11.0
Che	35000	16.5
STOP	—	—

6.3 Tasks

1. **Analysis.** List the inputs, the outputs and the processing rules of the problem. Identify which tasks require a selection structure and which require a loop. State the type of loop best suited (count-controlled or condition-controlled) and justify your choice.
2. **Pseudocode.** Write the complete algorithm in pseudocode. Use a `WHILE` loop with the sentinel `STOP`, nested `IF` (or `CASE`) for the grade, and initialise all counters and accumulators before the loop.
3. **Flowchart.** Draw the full flowchart of the algorithm, using the standard symbols (terminator, input/output, process, decision, connectors, arrows).
4. **Dry run.** Build a trace table with one column per variable (`name`, `fees`, `mark`, `status`, `grade`, `count`, `defaulters`, `totalFees`, `sumMarks`, `bestMark`, `bestName`) and trace the algorithm on the test data above. Predict the final summary report.
5. **Coding.** Translate the pseudocode into a working Python program.
6. **Testing.** Run the program with the test data. Compare the actual output with your dry-run prediction. Then test two boundary cases: a student with exactly 25 000 FCFA paid, and a student with an average of exactly 10.0. Write a short test report (5–8 lines) stating what was tested, the expected result, the actual result and any correction made.
7. **Extension (for fast groups).** Modify the program so that it also displays the percentage of defaulters and rejects any mark outside $[0, 20]$ by asking again (input validation loop).

WARNING **Common traps**

Initialising counters inside the loop resets them at every pass; forgetting to read the next name at the bottom of the loop creates an infinite loop; using > instead of >= at the boundary 25 000 misclassifies a student; and dividing by `count` when no student was entered causes a division by zero — protect the summary with a selection.

6.4 Model answer

1. Analysis.

Inputs: name (text), fees paid (real/integer), average mark (real). Outputs: per student, the fee status and the grade; at the end, the summary report. Processing: two selections per student (status, grade) inside a loop; four accumulators (`count`, `totalFees`, `sumMarks`, `bestMark`) and one counter (`defaulters`). The number of students is not known in advance, so a **condition-controlled** loop (`WHILE name <> "STOP"`) is required, with a priming read before the loop and a re-read at its end.

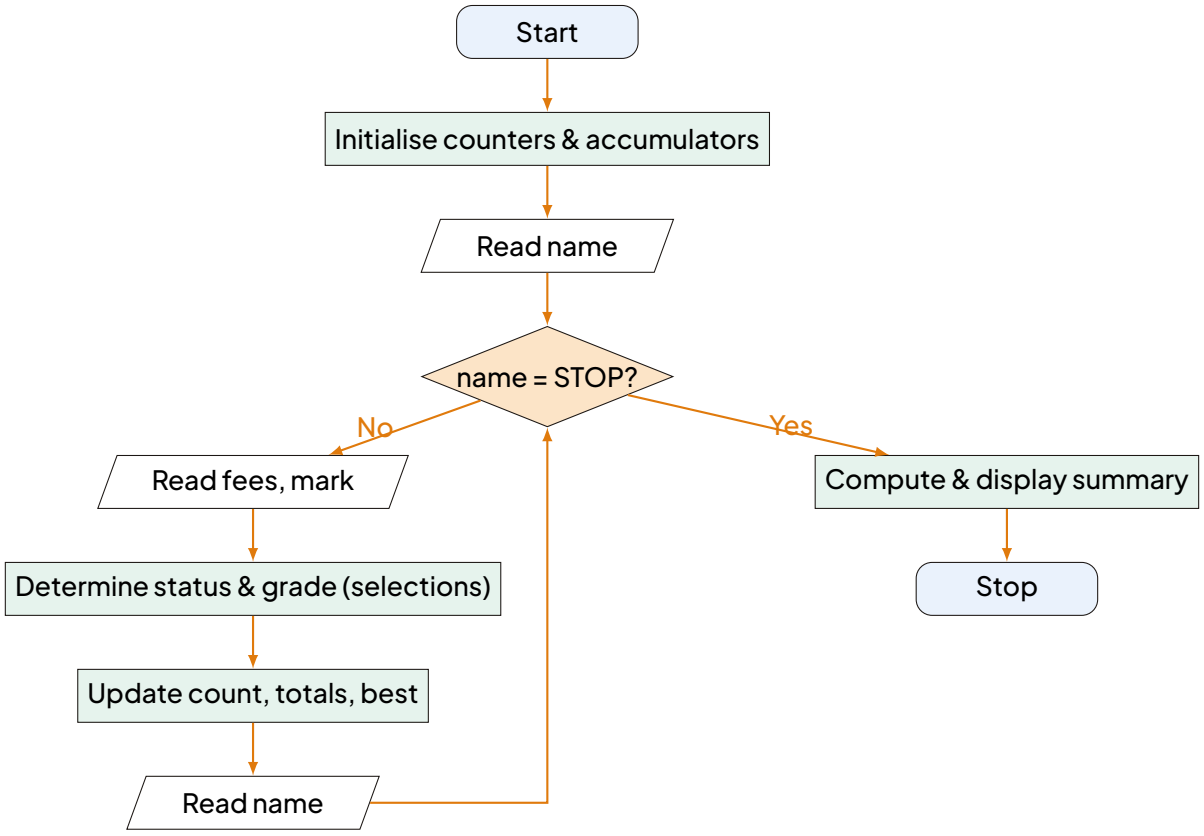
2. Pseudocode.

EXEMPLE **Algorithm FeesAndResults**

```
ALGORITHM FeesAndResults
CONSTANT FULL ← 50000, HALF ← 25000
count ← 0; defaulters ← 0
totalFees ← 0; sumMarks ← 0
bestMark ← -1; bestName ← ""
READ name
WHILE name <> "STOP" DO
  READ fees, mark
  IF fees = FULL THEN status ← "PAID UP"
  ELSE IF fees >= HALF THEN status ← "PART PAYMENT"
  ELSE status ← "DEFAULTER"; defaulters ← defaulters + 1
  ENDIF
  IF mark >= 16 THEN grade ← "A"
  ELSE IF mark >= 14 THEN grade ← "B"
  ELSE IF mark >= 12 THEN grade ← "C"
  ELSE IF mark >= 10 THEN grade ← "D"
  ELSE grade ← "F"
  ENDIF
  WRITE name, status, grade
  count ← count + 1
  totalFees ← totalFees + fees
  sumMarks ← sumMarks + mark
  IF mark > bestMark THEN bestMark ← mark; bestName ← name ENDIF
```

```
  READ name
ENDWHILE
IF count > 0 THEN
  WRITE count, defaulters, totalFees, sumMarks/count, bestMark, bestName
ELSE
  WRITE "No student entered"
ENDIF
END
```

3. Flowchart.



Flowchart of the fees and results mini-system (loop with sentinel, selections inside).

4. Dry run (trace table).

Name	Fees	Mark	Status	Grade	Count	Def.	TotalFees	SumMarks	BestMark	Be
Amina	50000	15.5	PAID UP	B	1	0	50000	15.5	15.5	Ar
Bih	20000	11.0	DEFAULTER	D	2	1	70000	26.5	15.5	Ar
Che	35000	16.5	PART PAY.	A	3	1	105000	43.0	16.5	Ch
STOP entered → loop ends; class average = 43.0/3 = 14.33										

Predicted summary: 3 students, 1 defaulter, total fees 105 000 FCFA, class average 14.33, best mark 16.5 (Che).

5. Code (Python).

EXEMPLE **fees_results.py**

```
FULL, HALF = 50000, 25000
count = defaulters = 0
totalFees = sumMarks = 0.0
bestMark = -1.0; bestName = ""
name = input("Name (STOP to end): ")
while name != "STOP":
  fees = float(input("Fees paid: "))
  mark = float(input("Average /20: "))
  if fees == FULL: status = "PAID UP"
  elif fees >= HALF: status = "PART PAYMENT"
  else: status = "DEFAULTER"; defaulters += 1
  if mark >= 16: grade = "A"
  elif mark >= 14: grade = "B"
  elif mark >= 12: grade = "C"
  elif mark >= 10: grade = "D"
  else: grade = "F"
  print(name, status, grade)
  count += 1; totalFees += fees; sumMarks += mark
  if mark > bestMark: bestMark, bestName = mark, name
  name = input("Name (STOP to end): ")
if count > 0:
  print("Students:", count, "Defaulters:", defaulters)
  print("Total fees:", totalFees, "FCFA")
  print("Class average:", round(sumMarks/count, 2))
  print("Best:", bestName, bestMark)
else:
  print("No student entered")
```

6. Test report (sample).

EXEMPLE **Test report**

Test 1 (test data): output matches the trace table exactly — 3 students, 1 defaulter, 105 000 FCFA, average 14.33, best Che 16.5. Test 2 (fees = 25 000): status PART PAYMENT as expected (boundary >= correct). Test 3 (mark = 10.0): grade D as expected. Test 4 (STOP entered first): "No student entered" displayed, no crash. No correction needed; the dry run predicted all results correctly.

EXERCISE 8 **Your turn**

Adapt the program for your own school: change the full fees to the real amount, add the percentage of defaulters, and validate that the mark lies in $[0, 20]$ using an inner `while` loop. Dry run your modified algorithm on two students before coding it.

7 METHODS AND WORKED EXAMPLES

8 ALGORITHMS AND PROGRAMMING

8.1 Methods and Worked Examples

This section presents the essential algorithmic techniques required for the GCE Advanced Level examination. Each method is structured as a step-by-step procedure followed by a fully worked examination-style question. Master the **dry run** (trace table) technique; it is your primary tool for verifying logic and earning method marks even if the final code has syntax errors.

METHOD **Constructing a Selection Structure (IF-THEN-ELSE)**

Objective: Implement decision making based on a condition.

1. Identify the condition: Write the Boolean expression that determines the path (e.g., `Mark >= 50`, `Age > 18 AND Nationality = "Cameroonian"`).
2. Define the TRUE branch: List the statements executed if the condition evaluates to `TRUE`.
3. Define the FALSE branch (ELSE): List the statements executed if the condition evaluates to `FALSE`. If no action is needed, the ELSE block can be omitted (IF-THEN).
4. Handle multiple mutually exclusive choices: Use `ELSE IF` ladder for ranges (e.g., Grade A, B, C, D, F). Order conditions carefully (usually highest to lowest or vice versa).
5. Nesting: An IF block can contain another IF block. Indent clearly to avoid **dangling else** ambiguity.

Key Reflex: Always test **boundary values** (e.g., exactly 50, exactly 18) during dry runs.

WORKED EXAMPLE 16 **Grading System with Validation**

Statement: Write an algorithm (pseudocode) that reads a student's `Mark` (integer, 0–100). Validate the input (reject if < 0 or > 100). If valid, output the grade according to the GCE scale: A (70–100), B (60–69), C (50–59), D (40–49), E (35–39), F (0–34). For invalid input, output "INVALID MARK".

SOLUTION

Detailed Solution:

1. Analysis:

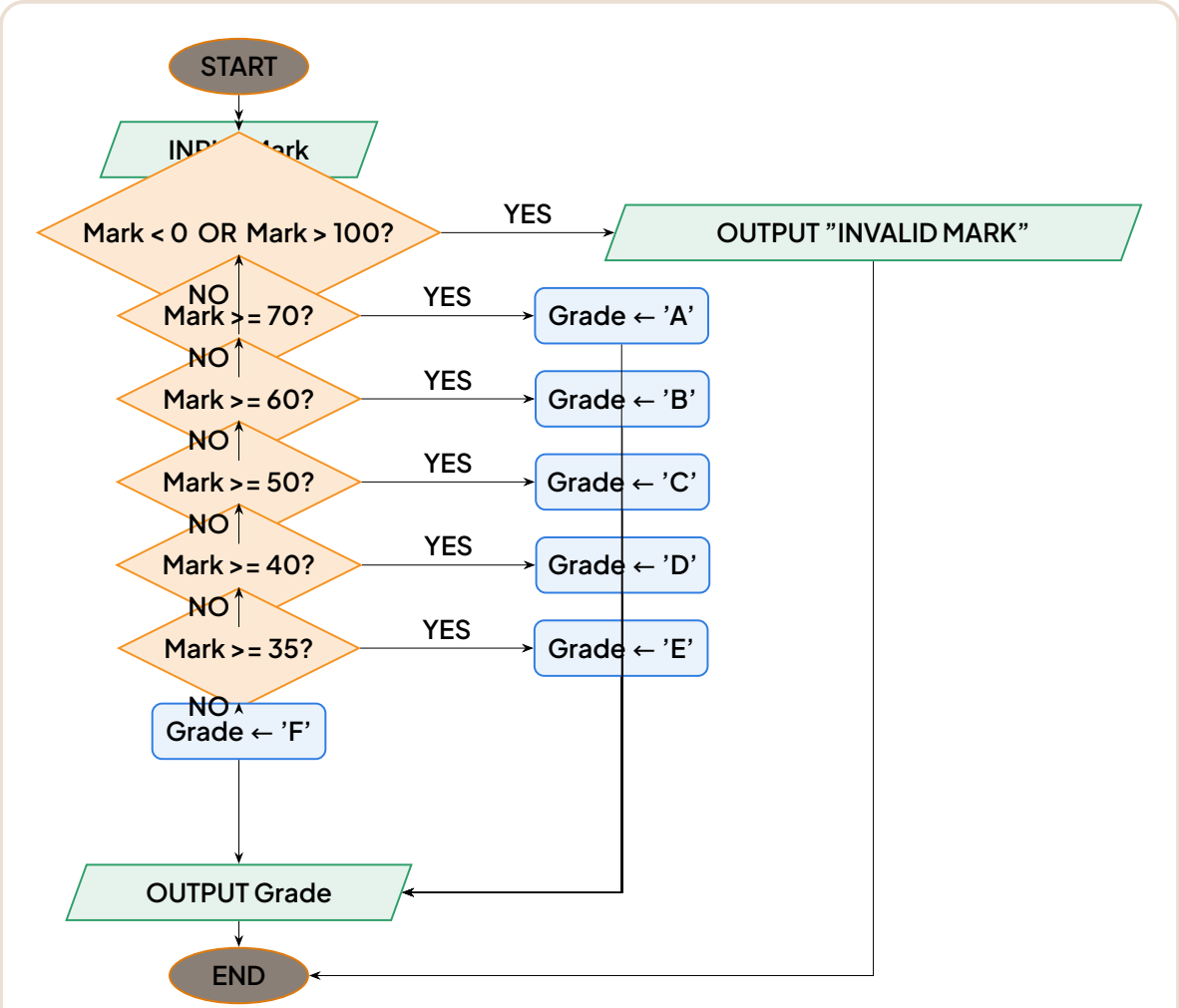
- Input: `Mark` (Integer).
- Process: Validation → Selection (Multi-way).
- Output: Grade (Character/String) or Error message.

2. Pseudocode Algorithm:

```
ALGORITHM GradeCalculator
VARIABLES
  Mark : INTEGER
  Grade : CHARACTER
BEGIN
  DISPLAY "Enter Mark (0-100): "
  READ Mark

  // Validation Selection
  IF Mark < 0 OR Mark > 100 THEN
    DISPLAY "INVALID MARK"
  ELSE
    // Multi-way Selection (Ordered High to Low)
    IF Mark >= 70 THEN
      Grade ← 'A'
    ELSE IF Mark >= 60 THEN
      Grade ← 'B'
    ELSE IF Mark >= 50 THEN
      Grade ← 'C'
    ELSE IF Mark >= 40 THEN
      Grade ← 'D'
    ELSE IF Mark >= 35 THEN
      Grade ← 'E'
    ELSE
      Grade ← 'F'
    END IF
    DISPLAY "Grade: ", Grade
  END IF
END
```

3. Flowchart Representation (TikZ):



Flowchart for Grading System with Validation

4. Dry Run (Trace Table): Test cases: `Mark = 75` (Valid, Grade A), `Mark = 105` (Invalid), `Mark = 34` (Valid, Grade F).

Step	Mark	Condition Checked	Result	Output
1	75	Mark < 0 OR > 100?	FALSE	–
2	75	Mark >= 70?	TRUE	Grade ← 'A'
3	75	–	–	OUTPUT "Grade: A"
1	105	Mark < 0 OR > 100?	TRUE	–
2	105	–	–	OUTPUT "INVALID MARK"
1	34	Mark < 0 OR > 100?	FALSE	–
2	34	Mark >= 70?	FALSE	–
3	34	Mark >= 60?	FALSE	–
4	34	Mark >= 50?	FALSE	–
5	34	Mark >= 40?	FALSE	–
6	34	Mark >= 35?	FALSE	–
7	34	ELSE	–	Grade ← 'F'
8	34	–	–	OUTPUT "Grade: F"

Examiner's Comment: Note the ordering of `ELSE IF` from highest to lowest. If you wrote `IF Mark >= 35 THEN ... ELSE IF Mark >= 70`, a mark of 75 would incorrectly yield Grade E. Always order range checks logically.

METHOD **Constructing Loop Structures (WHILE, REPEAT-UNTIL, FOR)**

Objective: Repeat a block of instructions efficiently.

1. Choose the loop type:
   - `WHILE Condition DO ... END WHILE`: **Pre-condition** loop. Zero or more iterations. Use when the number of iterations is unknown and zero is a valid count (e.g., reading until sentinel).
   - `REPEAT ... UNTIL Condition`: **Post-condition** loop. One or more iterations. Use when the loop body must execute at least once (e.g., menu display, input validation).

- `FOR Counter <- Start TO End [STEP Step] DO ... END FOR`: **Count-controlled** loop. Use when the exact number of iterations is known beforehand. Default step is 1.

2. Initialise: Set loop control variables (counters, accumulators, flags) before the loop starts.
3. Update: Ensure the loop variable changes inside the loop (increment counter, read next input, modify flag) to avoid **infinite loops**.
4. Termination: Verify the condition eventually becomes FALSE (WHILE/REPEAT) or the counter reaches the limit (FOR).
5. Nesting: Loops inside loops. Inner loop completes all its iterations for each single iteration of the outer loop. Use distinct counter variables (`i`, `j`).

Key Reflex: In a trace table, add a column for the Loop Condition evaluation (TRUE/FALSE) at the start/end of each iteration.

WORKED EXAMPLE 17 **Sentinel-Controlled Loop: Mobile Money Transaction Log**

Statement: A user enters transaction amounts (positive for credit, negative for debit) one by one. The session ends when the user enters `0` (sentinel). Write an algorithm to calculate and display:

1. Total number of transactions (excluding the sentinel).
2. Total Credits (sum of positive amounts).
3. Total Debits (sum of absolute values of negative amounts).
4. Final Balance (Credits - Debits).

SOLUTION

Detailed Solution:

1. Analysis:

- Unknown number of iterations → `WHILE` loop (pre-condition) is ideal because the user might enter 0 immediately (zero transactions).
- Sentinel value: `0`.
- Accumulators needed: `Count`, `SumCredit`, `SumDebit`.

2. Pseudocode Algorithm:

```
ALGORITHM MoMoStatement
VARIABLES
  Amount, Count, SumCredit, SumDebit, Balance : REAL
BEGIN
```

```
  // 1. Initialisation
  Count ← 0
  SumCredit ← 0
  SumDebit ← 0

  DISPLAY "Enter transaction amount (0 to end): "
  READ Amount     // Priming Read

  // 2. WHILE Loop (Pre-condition)
  WHILE Amount <> 0 DO
    Count ← Count + 1

    IF Amount > 0 THEN
      SumCredit ← SumCredit + Amount
    ELSE
      SumDebit ← SumDebit + (-Amount)   // Absolute value
    END IF

    DISPLAY "Enter next amount (0 to end): "
    READ Amount     // Update Read
  END WHILE

  // 3. Post-loop Calculations
  Balance ← SumCredit - SumDebit

  // 4. Output
  DISPLAY "Total Transactions: ", Count
  DISPLAY "Total Credits: ", SumCredit, " FCFA"
  DISPLAY "Total Debits: ", SumDebit, " FCFA"
  DISPLAY "Final Balance: ", Balance, " FCFA"
END
```

3. Dry Run (Trace Table): Input sequence: `5000, -2000, 10000, -500, 0`

Iteration	Amount Read	Condition (Amt<>0)	Count	SumCredit	SumDebit	Action
Init	–	–	0	0	0	Initialise
Priming	5000	TRUE	0	0	0	Enter Loop
1	5000	TRUE	1	5000	0	Credit
Read	-2000	–	1	5000	0	(Input -2000)
2	-2000	TRUE	2	5000	2000	Debit
Read	10000	–	2	5000	2000	(Input 10000)
3	10000	TRUE	3	15000	2000	Credit
Read	-500	–	3	15000	2000	(Input -500)
4	-500	TRUE	4	15000	2500	Debit
Read	0	–	4	15000	2500	(Input 0)
Check	0	FALSE	4	15000	2500	Exit Loop
Final	–	–	4	15000	2500	Balance = 12500

Examiner's Comment: The **Priming Read** (reading before the loop) and the **Update Read** (reading at the bottom of the loop) are mandatory for `WHILE` loops with sentinels. Missing either causes an infinite loop or skips the first/last value. `REPEAT...UNTIL` would require a flag or different logic to handle the "zero transactions" case correctly.

METHOD **Drawing Flowcharts to Standard (ISO 5807)**

Objective: Represent algorithms graphically using standard symbols.

1. Terminator (Oval): START, STOP, END.
2. Input/Output (Parallelogram): READ, INPUT, WRITE, PRINT, DISPLAY.
3. Process (Rectangle): Assignments (`x <- 5`), Calculations (`Sum <- A+B`), Function calls.
4. Decision (Diamond): Conditions (`A > B`, `Found = TRUE`). Two exit arrows labelled YES/TRUE and NO/FALSE.
5. Flow Lines (Arrows): Solid lines with arrowheads showing direction. Avoid crossing lines; use connectors (small circles with letters/numbers) for complex charts spanning pages.
6. Loop Representation:
   - `WHILE`: Decision diamond before process box. Arrow loops back to diamond.
   - `REPEAT-UNTIL`: Process box before decision diamond. Arrow loops back to process box.

- `FOR`: Often represented as a hexagon (preparation symbol) or a diamond with initialisation/increment annotation.

7. Subroutine/Predefined Process (Double-struck Rectangle): Call to a procedure/function.

Key Reflex: One entry, one exit per symbol (except Decision: one entry, two exits). No "spaghetti" arrows. Use a ruler (or TikZ grid).

WORKED EXAMPLE 18 **Flowchart for Nested Loop: Multiplication Table**

Statement: Draw a flowchart for an algorithm that prints the multiplication tables from 1 to 5 (i.e., 1 × 1 to 5 × 12). Use a `FOR` loop for the table number (outer) and a `FOR` loop for the multiplier (inner).

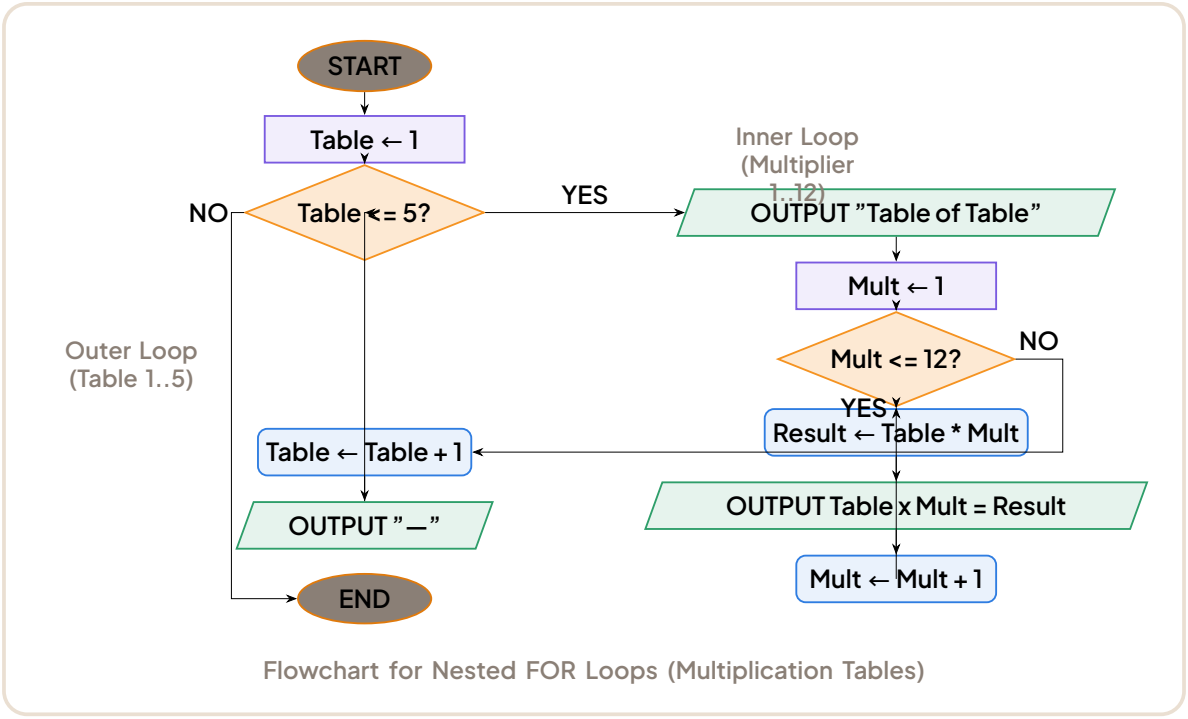
SOLUTION

Detailed Solution:

1. Pseudocode Logic (for reference):

```
FOR Table ← 1 TO 5 DO
  DISPLAY "Table of ", Table
  FOR Mult ← 1 TO 12 DO
    Result ← Table * Mult
    DISPLAY Table, " x ", Mult, " = ", Result
  END FOR
  DISPLAY "---"   // Separator
END FOR
```

2. Flowchart (TikZ):



Flowchart for Nested FOR Loops (Multiplication Tables)

Examiner's Comment: Note the use of the `prep` (hexagon/rectangle) shape for loop initialisation/increment steps to distinguish them from general processes. The inner loop (Mult) completes 12 cycles for each single cycle of the outer loop (Table). The flow returns from `Mult` increment directly to the `Mult <= 12?` diamond. The outer loop increment only happens after the inner loop finishes (NO branch of inner diamond).

METHOD **Dry Running (Trace Tables) – The Examiner's Favourite**

Objective: Simulate algorithm execution manually to find logic errors or produce output.

1. Set up columns: One column for every variable that changes, one for Output, and one for Condition Evaluations (True/False). Include a Line/Step column.
2. Execute line by line: Follow the control flow exactly. Do not "guess" the result; compute it.
3. Loops: Show every iteration as a new row (or group of rows). Write the condition check result explicitly (T/F).
4. Arrays/Lists: Use subscript columns (e.g., `A[0]`, `A[1]`, `A[2]`...) or a single column showing the whole array state `[1, 4, 2, ...]`.
5. Stop condition: Stop tracing when the algorithm terminates (END) or when the question asks for the state at a specific point.
6. Common Traps:

- Off-by-one errors in loop bounds (`TO` vs `TO ... STEP`, `<` vs `<=`).
- Integer division vs Real division (e.g., `5/2 = 2` in many pseudocode dialects, `2.5` in others. GCE usually assumes Real unless `DIV` / `MOD` used). Clarify assumption.
- Modulo operator: `MOD` gives remainder. `13 MOD 5 = 3`.
- Assignment (`<-`) vs Equality (`=`).

Key Reflex: Use a pencil. Draw a vertical line separating iterations. If a variable doesn't change, write - or carry forward the value (don't leave blank).

WORKED EXAMPLE 19 **Complex Dry Run: Array Processing with Flag**

Statement: Consider the following algorithm. `Data` is a 1D array of size 6 (indices 0 to 5).

```
ALGORITHM Mystery
VARIABLES
  Data : ARRAY[0..5] OF INTEGER
  i, Sum, Flag : INTEGER
BEGIN
  Data[0] ← 10; Data[1] ← -5; Data[2] ← 12; Data[3] ← 0; Data[4] ← 8; Data[5] ←
-3
  Sum ← 0
  Flag ← 0
  FOR i ← 0 TO 5 DO
    IF Data[i] > 0 THEN
      Sum ← Sum + Data[i]
      IF Data[i] MOD 2 = 0 THEN
        Flag ← Flag + 1
      END IF
    END IF
  END FOR
  DISPLAY Sum, Flag
END
```

Complete the trace table for the execution of this algorithm.

SOLUTION

Detailed Solution:

1. Initialisation State:

- `Data = [10, -5, 12, 0, 8, -3]`
- `Sum = 0, Flag = 0`

2. Trace Table: Columns: `i, Data[i], Data[i] > 0?, Data[i] MOD 2 = 0?, Sum, Flag, Output`.

i	Data[i]	> 0?	Even? (MOD 2=0)	Sum (before)	Sum (after)	Flag (after)	Notes
Init	–	–	–	0	0	0	Start
0	10	TRUE	TRUE (10%2=0)	0	10	1	Add 10, Inc Fla
1	-5	FALSE	–	10	10	1	Skip
2	12	TRUE	TRUE (12%2=0)	10	22	2	Add 12, Inc Fla
3	0	FALSE	–	22	22	2	0 is not > 0
4	8	TRUE	TRUE (8%2=0)	22	30	3	Add 8, Inc Flag
5	-3	FALSE	–	30	30	3	Skip
End	–	–	–	30	30	3	Loop Ends

3. Final Output: `DISPLAY 30, 3`
Examiner's Comment:

- Boundary Check: Index 0 to 5 inclusive (6 iterations). `FOR i <- 0 TO 5` implies inclusive bounds in standard pseudocode.
- Condition Nuance: `Data[3] = 0`. Condition `Data[i] > 0` is FALSE. Zero is not positive. This is a classic trap.
- MOD Logic: `10 MOD 2 = 0, 12 MOD 2 = 0, 8 MOD 2 = 0`. All positive numbers here happen to be even. `Flag` counts positive even numbers.
- Presentation: Show "Before" and "After" for accumulators (`Sum`, `Flag`) to demonstrate the update clearly to the examiner.

METHOD **Writing Code: Pseudocode to Python 3 (GCE Standard)**

Objective: Translate algorithmic logic into executable Python 3 code. GCE expects standard, readable syntax.

1. Variables: No declaration needed. Dynamic typing. `mark = int(input())`.
2. Selection: `if condition: ... elif condition: ... else: ...` Indentation (4 spaces) is mandatory and defines blocks. No `END IF`. Colon `:` required after condition.
3. Loops:
   - `while condition:` (Pre-condition).
   - `for i in range(start, stop, step):` Note: `range(stop)` goes 0 to stop-1. `range(1, 13)` gives 1..12. `range(5, 0, -1)` gives 5,4,3,2,1.
   - No native `REPEAT-UNTIL`. Simulate: `while True: ... if condition: break`.
4. Input/Output: `input("Prompt")` returns `str`. Cast: `int()`, `float()`. `print(f"Value:`

`{var}")` (f-strings preferred) or `print("Value:", var)`.

5. Arrays/Lists: `list = []` (dynamic). `list.append(val)`. Indexing 0-based. `len(list)` for size.
6. Operators: `//` (Integer division), `%` (Modulo), `**` (Power), `and`, `or`, `not` (Boolean).
7. Functions/Procedures: `def name(params): ... return val`. Call by object reference.

Key Reflex: In the exam, if asked for "Program Code", write Python. If asked for "Algorithm/Pseudocode", write structured English/pseudocode (indented, keywords capitalised). Do not mix them.

WORKED EXAMPLE 20 **Translation: Sentinel Loop to Python**

Statement: Convert the "Mobile Money Statement" algorithm (Worked Example 2) into a complete Python 3 program. Add a function `get_amount()` that handles input validation (ensuring the user enters a valid number) and returns the float value.

SOLUTION

Detailed Solution:
1. Modular Design: Separate input logic (robustness) from business logic (calculation).
2. Python 3 Code:

```
def get_amount(prompt: str) -> float:
  """ Repeatedly prompts user until a valid float is entered.
  Returns the float value. """
  while True:
    try:
      value = float(input(prompt))
      return value
    except ValueError:
      print("Invalid input. Please enter a numeric value.")

def main():
  # Initialisation
  count = 0
  sum_credit = 0.0
  sum_debit = 0.0

  # Priming Read
  amount = get_amount("Enter transaction amount (0 to end): ")

  # WHILE Loop (Sentinel Controlled)
  while amount != 0:
    count += 1

    if amount > 0:
      sum_credit += amount
```

```
    else:
      # amount is negative, add absolute value to debit
      sum_debit += abs(amount)

    # Update Read
    amount = get_amount("Enter next amount (0 to end): ")

  # Calculations
  balance = sum_credit - sum_debit

  # Output (Formatted to 2 decimal places for currency)
  print(f"\n--- MoMo Statement ---")
  print(f"Total Transactions : {count}")
  print(f"Total Credits : {sum_credit:,.2f} FCFA")
  print(f"Total Debits : {sum_debit:,.2f} FCFA")
  print(f"Final Balance : {balance:,.2f} FCFA")

if __name__ == "__main__":
  main()
```

3. Key Python Features Demonstrated:

- `try...except ValueError`: Robust input handling (catches non-numeric input like "abc").
- `while True: ... break`: Idiomatic replacement for `REPEAT-UNTIL` inside `get_amount`.
- `abs()`: Built-in absolute value.
- `f-strings` with `:,.2f`: Formats float with comma thousand separators and 2 decimal places (e.g., `15,000.00`).
- `if __name__ == "__main__"`: Best practice for script entry point.

Examiner's Comment: GCE marking schemes award marks for: correct loop structure (`while amount != 0`), correct accumulation (+=), correct sentinel handling (priming + update read), and correct output formatting. The function `get_amount` demonstrates "Writing Code 3" (modularity/procedures) syllabus requirement.

METHOD **Integration Activity: Top-Down Design & Modular Programming**

Objective: Decompose a complex problem into modules (Main algorithm + Procedures/Functions) using Structure Charts.

1. Identify the Main Goal: What is the single output or result?
2. Decompose (Top-Down): Break into 3–5 major sub-tasks (Input, Process, Output, Validation, File Handling). Each becomes a **Procedure** (no return value, side effect) or **Function** (returns a value).

3. Define Interfaces: For each module, specify Parameters (IN, OUT, INOUT) and Return Type.
4. Data Dictionary: List all global/local variables, types, and scope.
5. Refine Modules: Write pseudocode for each module independently. Use `CALL ProcedureName(Args)` or `Var <- FunctionName(Args)`.
6. Parameter Passing:
   - `BY VALUE` (Default for primitives in Python/pseudocode): Copy passed. Original unchanged.
   - `BY REFERENCE` (Arrays/Objects, or explicit `REF` keyword): Alias passed. Original modified.
7. Structure Chart: Draw hierarchy (Rectangles = Modules, Arrows = Calls, Data couples = Arrows with circles).

Key Reflex: High Cohesion (module does ONE thing), Low Coupling (modules share minimal data, prefer parameters over globals).

WORKED EXAMPLE 21 **Integrated System: Student Result Management**

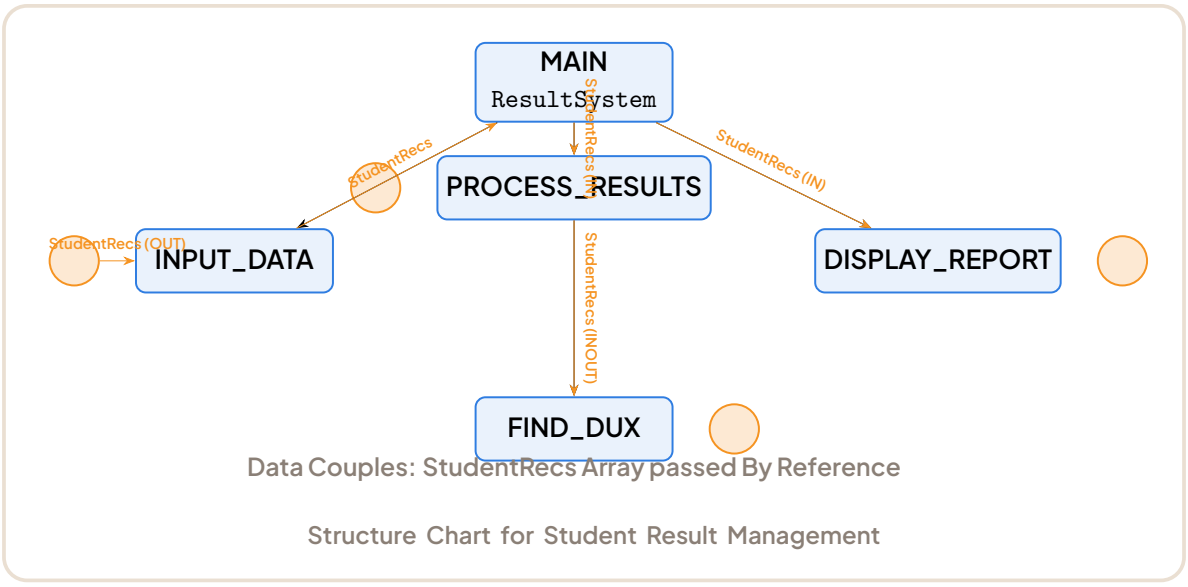
Statement: Design a modular algorithm for a class teacher. The system must:

1. Allow entry of marks for 5 students, 3 subjects each (Math, English, ICT). Marks 0–100.
2. Calculate each student's Average and Grade (A:>=70, B:>=60, C:>=50, D:>=40, E:>=35, F:<35).
3. Identify the Student with the Highest Average (Dux).
4. Display a formatted Report Card for all students.

Provide: (a) Structure Chart, (b) Data Dictionary, (c) Main Algorithm + Module Pseudocode.

SOLUTION

Detailed Solution:
(a) Structure Chart (TikZ):



(b) Data Dictionary:

Identifier	Type	Scope	Description
MAX_STUDENTS	CONSTANT INT	Global	5
MAX_SUBJECTS	CONSTANT INT	Global	3
StudentRecs	ARRAY[0..4] OF RECORD	Global (Main)	Array of Student Records
.Name	STRING	Record Field	Student Name
.Marks	ARRAY[0..2] OF INT	Record Field	Math, Eng, ICT
.Average	REAL	Record Field	Calculated Average
.Grade	CHAR	Record Field	Calculated Grade
DuxIndex	INT	Local (Main)	Index of Top Student
i, j	INT	Local (Modules)	Loop Counters
Sum, Avg	REAL	Local (Process)	Temp Calculation Vars

(c) Algorithms:
MAIN MODULE

```
ALGORITHM ResultSystem
CONSTANTS
  MAX_STUDENTS = 5, MAX_SUBJECTS = 3
VARIABLES
  StudentRecs : ARRAY[0..MAX_STUDENTS-1] OF StudentRecord
  DuxIndex : INTEGER
BEGIN
  CALL InputData(StudentRecs)     // Pass By Reference (Array)
  CALL ProcessResults(StudentRecs)   // Pass By Reference
  DuxIndex ← FindDux(StudentRecs)   // Function Returns Index
```

```
  CALL DisplayReport(StudentRecs, DuxIndex)   // Pass By Reference
END
```

MODULE: InputData (Procedure)

```
PROCEDURE InputData(REF Students : ARRAY OF StudentRecord)
VARIABLES i, j : INTEGER; Mark : INTEGER
BEGIN
  FOR i ← 0 TO MAX_STUDENTS - 1 DO
    DISPLAY "Enter Name for Student ", i+1, ": "
    READ Students[i].Name
    FOR j ← 0 TO MAX_SUBJECTS - 1 DO
      REPEAT
        DISPLAY " Subject ", j+1, " Mark (0-100): "
        READ Mark
        IF Mark < 0 OR Mark > 100 THEN
          DISPLAY "Invalid! Enter 0-100."
        END IF
      UNTIL Mark >= 0 AND Mark <= 100
      Students[i].Marks[j] ← Mark
    END FOR
  END FOR
END PROCEDURE
```

MODULE: ProcessResults (Procedure)

```
PROCEDURE ProcessResults(REF Students : ARRAY OF StudentRecord)
VARIABLES i, j : INTEGER; Sum : REAL
BEGIN
  FOR i ← 0 TO MAX_STUDENTS - 1 DO
    Sum ← 0
    FOR j ← 0 TO MAX_SUBJECTS - 1 DO
      Sum ← Sum + Students[i].Marks[j]
    END FOR
    Students[i].Average ← Sum / MAX_SUBJECTS

    // Grade Function Logic Inline
    IF Students[i].Average >= 70 THEN Students[i].Grade ← 'A'
    ELSE IF Students[i].Average >= 60 THEN Students[i].Grade ← 'B'
    ELSE IF Students[i].Average >= 50 THEN Students[i].Grade ← 'C'
    ELSE IF Students[i].Average >= 40 THEN Students[i].Grade ← 'D'
    ELSE IF Students[i].Average >= 35 THEN Students[i].Grade ← 'E'
    ELSE Students[i].Grade ← 'F'
    END IF
  END FOR
END PROCEDURE
```

MODULE: FindDux (Function)

```
FUNCTION FindDux(Students : ARRAY OF StudentRecord) RETURNS INTEGER
VARIABLES i, BestIdx : INTEGER; BestAvg : REAL
BEGIN
  BestIdx ← 0
  BestAvg ← Students[0].Average
```

```
  FOR i ← 1 TO MAX_STUDENTS - 1 DO
    IF Students[i].Average > BestAvg THEN
      BestAvg ← Students[i].Average
      BestIdx ← i
    END IF
  END FOR
  RETURN BestIdx
END FUNCTION
```

MODULE: DisplayReport (Procedure)

```
PROCEDURE DisplayReport(Students : ARRAY OF StudentRecord, DuxIdx : INTEGER)
VARIABLES i, j : INTEGER
BEGIN
  DISPLAY "================================================"
  DISPLAY " CLASS REPORT CARD - UPPER SIXTH "
  DISPLAY "================================================"
  DISPLAY "Name | Math | Eng | ICT | Avg | Grade"
  DISPLAY "------------------------------------------------"
  FOR i ← 0 TO MAX_STUDENTS - 1 DO
    // Formatted Output Simulation
    DISPLAY Students[i].Name, " | ",
      Students[i].Marks[0], " | ",
      Students[i].Marks[1], " | ",
      Students[i].Marks[2], " | ",
      Students[i].Average (Format 1 dec), " | ",
      Students[i].Grade
    IF i == DuxIdx THEN
      DISPLAY " * DUx AWARD *"
    END IF
  END FOR
  DISPLAY "================================================"
END PROCEDURE
```

Examiner's Comment: This integration example covers all syllabus lessons: Selection (Validation, Grading), Loops (Nested For, Repeat-Until), Flowcharts (Structure Chart), Dry Running (trace each module), Writing Code (Modular, Parameters By Ref/Value). In the exam, you may be asked to write one specific module (e.g., "Write the `FindDux` function"). Practice writing modules in isolation.

METHOD **Advanced Trace: Nested Loops with Array Manipulation (Sorting/Search)**

Objective: Trace algorithms that modify array order (Bubble Sort, Linear Search, Binary Search).

1. Draw the Array State: Create columns for each index `A[0]`, `A[1]`... `A[N-1]`. Update cells only when values change.
2. Track Pointers/Indices: Columns for `i`, `j`, `low`, `high`, `mid`, `temp`.

3. Show Swaps Explicitly: When `Temp <- A[j]; A[j] <- A[j+1]; A[j+1] <- Temp`, show the three steps or the "Before/After" row.
4. Loop Invariants: Note what is guaranteed at the start/end of each outer loop pass (e.g., "Largest element bubbled to end").
5. Early Exit: Trace the `Break` or `Return` condition carefully.

Key Reflex: For Bubble Sort, the inner loop range shrinks (`0 to N-2-i`). For Binary Search, the array must be sorted. Trace `mid` calculation: `mid = (low + high) // 2` (integer division).

WORKED EXAMPLE 22 **Trace: Bubble Sort Pass Optimization**

Statement: Trace the following Bubble Sort algorithm for the array `A = [5, 1, 4, 2, 8]`. Show the array state after each complete pass of the outer loop and the total number of swaps. The algorithm includes an optimization flag `Swapped`.

```
ALGORITHM BubbleSortOpt
VARIABLES A: ARRAY[0..4] OF INT; i, j, Temp: INT; Swapped: BOOLEAN
BEGIN
  A ← [5, 1, 4, 2, 8]
  FOR i ← 0 TO 3 DO     // Max N-1 passes
    Swapped ← FALSE
    FOR j ← 0 TO 3 - i DO   // Range shrinks
      IF A[j] > A[j+1] THEN
        Temp ← A[j]
        A[j] ← A[j+1]
        A[j+1] ← Temp
        Swapped ← TRUE
      END IF
    END FOR
    IF Swapped = FALSE THEN
      EXIT FOR     // Early termination
    END IF
  END FOR
  DISPLAY A
END
```

SOLUTION

Detailed Solution:
Initial State: `A = [5, 1, 4, 2, 8], Swaps = 0`
Pass 1 (i=0, j=0 to 3):

j	Compare	Swap?	A[0]	A[1]	A[2]	A[3]	A[4]
Start	–	–	5	1	4	2	8
0	5 > 1	YES	1	5	4	2	8
1	5 > 4	YES	1	4	5	2	8
2	5 > 2	YES	1	4	2	5	8
3	5 > 8	NO	1	4	2	5	8

`Swapped = TRUE`. Swaps this pass: 3. Total: 3.
Pass 2 (i=1, j=0 to 2):

j	Compare	Swap?	A[0]	A[1]	A[2]	A[3]	A[4]
Start	–	–	1	4	2	5	8
0	1 > 4	NO	1	4	2	5	8
1	4 > 2	YES	1	2	4	5	8
2	4 > 5	NO	1	2	4	5	8

`Swapped = TRUE`. Swaps this pass: 1. Total: 4.
Pass 3 (i=2, j=0 to 1):

j	Compare	Swap?	A[0]	A[1]	A[2]	A[3]	A[4]
Start	–	–	1	2	4	5	8
0	1 > 2	NO	1	2	4	5	8
1	2 > 4	NO	1	2	4	5	8

`Swapped = FALSE`. EXIT FOR triggered.
Final Output: `[1, 2, 4, 5, 8]` Total Swaps: 4. Total Passes Executed: 3 (out of max 4).
Examiner's Comment: The optimization `EXIT FOR` (or `BREAK`) saves the 4th pass. In a trace table question, you must show the `Swapped` flag check. If asked "How many comparisons?", count the `IF` evaluations: Pass 1 (4) + Pass 2 (3) + Pass 3 (2) = 9 comparisons.

METHOD **Writing Code 3: File Handling & Data Persistence (Text/CSV)**

Objective: Read from and write to external files (simulating databases/logs).

1. Modes: `'r'` (Read), `'w'` (Write - overwrites!), `'a'` (Append), `'r+'` (Read/Write).
2. Context Manager: `with open("file.txt", "mode") as f:` → Auto-closes file (best practice).
3. Writing: `f.write(string)`. Must convert numbers to string `str(val)` or use f-strings. Add `\n` for new lines.

4. Reading:
   - `f.read()`: Whole file as one string.
   - `f.readline()`: One line (keeps `\n`).
   - `f.readlines()`: List of lines.
   - `for line in f:`: Iterate lines (memory efficient).
5. CSV Module: `import csv. writer.writerow(list), reader` yields lists. Handles commas/quotes automatically.
6. Exceptions: `FileNotFoundError, PermissionError, ValueError` (parsing). Use `try...except`.

Key Reflex: Always close files (or use `with`). In pseudocode: `OPEN "file" FOR INPUT/OUTPUT/APPEND AS #1, READ #1, Var, WRITE #1, Var, CLOSE #1.`

WORKED EXAMPLE 23 **File Handling: Saving/Loading Student Records (CSV)**

Statement: Write two Python functions:

1. `save_records(filename, students)`: Saves list of dictionaries `[{'name': 'John', 'avg': 75.5, 'grade': 'A'}, ...]` to CSV with header.
2. `load_records(filename)`: Reads CSV, reconstructs list of dictionaries. Handles missing file gracefully.

SOLUTION

Detailed Solution:
Python 3 Implementation:

```
import csv
import os

def save_records(filename: str, students: list) -> bool:
  """ Saves student records to CSV.
  Returns True on success, False on error. """
  # Define field order for consistency
  fields = ['name', 'avg', 'grade']

  try:
    # newline='' prevents blank rows on Windows
    with open(filename, 'w', newline='', encoding='utf-8') as f:
      writer = csv.DictWriter(f, fieldnames=fields)
      writer.writeheader()   # Writes: name,avg,grade
      for student in students:
        # Ensure data types are strings for CSV writer
        row = {
```

```
            'name': student['name'],
            'avg': f"{student['avg']:.2f}",   # Format float
            'grade': student['grade']
          }
          writer.writerow(row)
      print(f"Successfully saved {len(students)} records to {filename}")
      return True
    except (IOError, OSError, PermissionError) as e:
      print(f"Error writing file: {e}")
      return False

def load_records(filename: str) -> list:
    """ Loads student records from CSV.
    Returns list of dicts (empty if file not found or error). """
    students = []
    if not os.path.exists(filename):
      print(f"File {filename} not found. Starting with empty records.")
      return students

    try:
      with open(filename, 'r', newline='', encoding='utf-8') as f:
        reader = csv.DictReader(f)   # Assumes header exists
        for row in reader:
          # Convert types back from strings
          student = {
            'name': row['name'],
            'avg': float(row['avg']),
            'grade': row['grade']
          }
          students.append(student)
      print(f"Loaded {len(students)} records from {filename}")
      return students
    except (IOError, OSError, KeyError, ValueError) as e:
      print(f"Error reading/parsing file: {e}")
      return []   # Return empty on corruption

# --- Integration Test ---
if __name__ == "__main__":
    # Sample Data (from Integration Example)
    class_data = [
      {'name': 'Amina', 'avg': 82.33, 'grade': 'A'},
      {'name': 'Paul', 'avg': 55.00, 'grade': 'C'},
      {'name': 'Grace', 'avg': 38.50, 'grade': 'E'}
    ]

    fname = "results_upper6.csv"

    # Save
    save_records(fname, class_data)

    # Load back
    loaded = load_records(fname)
```

```
# Verify
print("\n--- Loaded Data ---")
for s in loaded:
  print(f"{s['name']:10} | Avg: {s['avg']:5.2f} | Grade: {s['grade']}")
```

Examiner's Comment:

- `csv.DictWriter` / `DictReader` map dictionaries to rows by header name, making code robust to column order changes.
- `newline=''` is a specific Python 3 requirement for CSV to prevent blank lines on Windows.
- `encoding='utf-8'` ensures Cameroonian names with accents (e.g., `Étienne`, `Ngön`) are saved correctly.
- Error handling (`try...except`) is crucial for "Writing Code 3" marks (robustness).
- In Pseudocode (Exam): `OPEN "results.csv" FOR OUTPUT AS #1, WRITE #1, Name, ",", Avg, ",", Grade, CLOSE #1.`

KEY POINTS **GCE Exam Checklist for ICT26**

- Selection: Master `IF-ELSEIF-ELSE` ladder ordering. Boundary testing (>, >=).
- Loops: Distinguish `WHILE` (0+ times), `REPEAT` (1+ times), `FOR` (Count). Priming/Update reads for sentinels.
- Flowcharts: Correct symbols (Diamond=Decision, Parallelogram=I/O). Two labelled exits from Diamond. Connectors for off-page.
- Dry Run: Trace table with columns for ALL changing variables + Condition column. Show every iteration.
- Python Code: Indentation (4 spaces). `range(start, stop, step)` exclusivity. `//` vs `/`. `and/or/not` vs `&&/||/!`. f-strings for output.
- Modularity: Structure Chart, Data Dictionary, Parameter Passing (By Val vs By Ref). Cohesion/Coupling.
- Files: `with open(...)`, `csv` module, Exception handling.
- Common Algorithms: Linear Search, Binary Search (Sorted!), Bubble Sort (Swaps/Comparisons), Max/Min finding.

9 EXERCISES

9.1 I check my knowledge

EXERCISE 9 **Multiple choice questions**

For each question, choose the single correct answer.

1. In the pseudocode statement IF $x > 10$ THEN … ELSE … ENDIF, the ELSE part is executed when:

   (a) $x > 10$ (b) $x \geq 10$ (c) $x \leq 10$ (d) $x = 10$ only.

2. Which loop structure is most appropriate when the number of repetitions is known in advance?

   (a) WHILE (b) REPEAT … UNTIL (c) FOR (d) IF.

3. In a flowchart, a diamond-shaped symbol represents:

   (a) the start of the algorithm (b) an input/output operation (c) a processing step (d) a decision (test).

4. A dry run of an algorithm is used to:

   (a) translate the algorithm into machine language (b) check the logic by tracing variable values by hand (c) draw the flowchart automatically (d) speed up the execution of the program.

EXERCISE 10 **True or false — justify**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. The body of a WHILE loop is always executed at least once.
2. A CASE structure can always be replaced by a series of nested IF … ELSE IF statements.
3. In a trace table, each column corresponds to one variable or output of the algorithm.
4. A FOR loop in which the counter is modified inside the loop body is good programming practice.

EXERCISE 11 **Definitions**

Define each of the following terms in one or two sentences, and give a short pseudocode example for (a), (b) and (c):

1. selection (conditional) control structure;
2. loop (iterative) control structure;
3. sentinel value;
4. dry run (trace) of an algorithm;
5. infinite loop.

EXERCISE 12 **Direct evaluation**

Consider the following pseudocode fragment:

```
a ← 7;   b ← 3
IF a mod b = 1 THEN c ← a + b ELSE c ← a - b ENDIF
FOR i FROM 1 TO 4 DO s ← s + i ENDFOR    (with s ← 0 before the loop)
```

1. Give the value of c after the IF statement.
2. Give the value of s after the FOR loop.
3. How many times is the body of the FOR loop executed?

9.2 I practise

EXERCISE 13 **Ascending order of three numbers**

Write the pseudocode and draw the flowchart of an algorithm that reads three distinct real numbers a, b, c and displays them in ascending order. Use only simple two-way selections (IF … THEN … ELSE).

EXERCISE 14 **Dry run of a WHILE loop**

The following algorithm is supposed to compute the product of the even numbers from 2 to 10:

```
p ← 1
n ← 2
WHILE n ≤ 10 DO
    p ← p × n
    n ← n + 2
ENDWHILE
Display p
```

1. Copy and complete the trace table with columns n, condition $n \leq 10$ (true/false), and p.
2. State the value displayed by the algorithm.
3. State how many times the loop body is executed.

EXERCISE 15 **From nested IF to CASE**

A grading algorithm uses nested selections:

```
Read m    (a mark out of 100)
IF m < 50 THEN grade ← "Fail"
ELSE IF m < 65 THEN grade ← "Pass"
ELSE IF m < 80 THEN grade ← "Credit"
ELSE grade ← "Distinction" ENDIF
Display grade
```

1. Rewrite this algorithm using a CASE structure based on the value of m div 10 (integer division), adding a default case for invalid marks.
2. State the output of the algorithm for the inputs $m = 45$, $m = 70$ and $m = 95$.

EXERCISE 16 **Hunting the sentinel error**

The following algorithm should compute the average of marks entered by a teacher, stopping when -1 is typed:

```
s ← 0;   n ← 0
Read m
WHILE m ≠ -1 DO
    Read m
    s ← s + m
    n ← n + 1
ENDWHILE
Display s / n
```

1. Dry run this algorithm with the input sequence 60, 40, -1 and give the value displayed.
2. Explain why the result is wrong (two distinct errors).
3. Rewrite the algorithm correctly so that the -1 is neither added nor counted, and no mark is lost.

9.3 I prepare for the GCE

EXERCISE 17 **Multiplication table — FOR and WHILE**

(8 marks)

1. Write a complete program (in a language of your choice, clearly named) that reads an integer N and displays its multiplication table from 1 to 12, using a FOR loop. (4 marks)
2. Modify your program so that the table stops at an upper limit L entered by the user, using a WHILE loop instead of the FOR loop. (3 marks)
3. State one advantage of the FOR loop over the WHILE loop in the first version. (1 mark)

EXERCISE 18 **Number-guessing game**

(10 marks) A program picks a secret integer between 1 and 100. The user repeatedly proposes a number; after each attempt the program displays "too high", "too low" or "correct", and stops when the secret number is found.

1. Draw the complete flowchart of this program, using the correct standard symbols. (5 marks)
2. Write the corresponding pseudocode, using a REPEAT … UNTIL or WHILE loop and a selection structure. (3 marks)

3. Dry run your pseudocode with secret number 37 and attempts 50, 25, 37: give the sequence of messages displayed. (2 marks)

EXERCISE 19 **Class marks statistics — GCE style**

(12 marks) Consider the following pseudocode, intended to process the marks of 30 students:

```
pass ← 0;   fail ← 0;   max ← 0;   sum ← 0
FOR i FROM 1 TO 30 DO
    Read m
    IF m ≥ 50 THEN pass ← pass + 1 ELSE fail ← fail + 1 ENDIF
    IF m > max THEN max ← m ENDIF
    sum ← sum + m
ENDFOR
avg ← sum / 30
Display pass, fail, max, avg
```

1. State the purpose of each of the four variables $pass$, $fail$, max and sum. (2 marks)
2. How many times is the loop body executed? (1 mark)
3. Dry run the algorithm for the first three marks 45, 72, 50 only (assume the loop stops after 3 students and $avg \leftarrow sum/3$): complete a trace table and state the values displayed. (4 marks)
4. Explain why initialising max to 0 can give a wrong result, and propose a correction. (2 marks)
5. Identify the type of control structures used in this algorithm and classify the error found in (d) as a syntax, runtime or logic error, justifying your answer. (3 marks)

10 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. (c) $x \leq 10$. The `ELSE` part is executed when the condition $x > 10$ is false, i.e. when $x \leq 10$.
2. (c) FOR. A `FOR` loop is designed for a known number of iterations (counter-controlled loop).
3. (d) a decision (test). In standard flowcharting, a diamond (rhombus) represents a decision point where a condition is tested.
4. (b) check the logic by tracing variable values by hand. A dry run (or trace) is a manual step-by-step execution recording variable values to verify correctness.

SOLUTION **Exercise 2 — True or false — justify**

1. False. A `WHILE` loop tests the condition before each iteration; if the condition is initially false, the body is never executed.
2. True. A `CASE` (or `SWITCH`) structure is syntactic sugar for a chain of `IF …ELSE IF …ELSE` where the same variable is compared to constant values. Any `CASE` can be rewritten as nested `IF` statements.
3. True. In a trace table each column tracks one variable (or expression/output) and each row corresponds to a step (line) of the algorithm.
4. False. Modifying the loop counter inside a `FOR` loop body is considered bad practice because it makes the loop behaviour unpredictable and harder to read; it can lead to logic errors or infinite loops.

SOLUTION **Exercise 3 — Definitions**

1. Selection (conditional) control structure: A structure that chooses between two or more alternative paths of execution based on the evaluation of a Boolean condition. Example:

```
IF age >= 18 THEN
  status <- "Adult"
ELSE
  status <- "Minor"
ENDIF
```

2. Loop (iterative) control structure: A structure that repeats a block of statements while a condition holds (pre-test), until a condition becomes true (post-test), or for a fixed number of iterations (counter-controlled). Example:

```
FOR i FROM 1 TO 10 DO
  DISPLAY i
ENDFOR
```

3. Sentinel value: A special input value that signals the end of data entry in a loop; it is not processed as part of the data. Example: Using -1 to stop reading positive marks.
4. Dry run (trace) of an algorithm: A manual simulation of the algorithm step by step, recording the values of all variables after each statement to verify the logic and detect errors.
5. Infinite loop: A loop whose terminating condition never becomes true, causing the loop to repeat indefinitely (unless interrupted externally). Example:

```
WHILE true DO
  DISPLAY "Hello"
ENDWHILE
```

SOLUTION **Exercise 4 — Direct evaluation**

1. $a = 7, b = 3$. $a \bmod b = 7 \bmod 3 = 1$. The condition $a \bmod b = 1$ is true, so $c \leftarrow a + b = 7 + 3 = 10$. Value of c: 10.
2. The `FOR` loop runs for $i = 1, 2, 3, 4$. Initially $s = 0$. $i = 1$: $s = 0 + 1 = 1$ $i = 2$: $s = 1 + 2 = 3$ $i = 3$: $s = 3 + 3 = 6$ $i = 4$: $s = 6 + 4 = 10$ Value of s after the loop: 10.
3. The loop runs from 1 to 4 inclusive: 4 times.

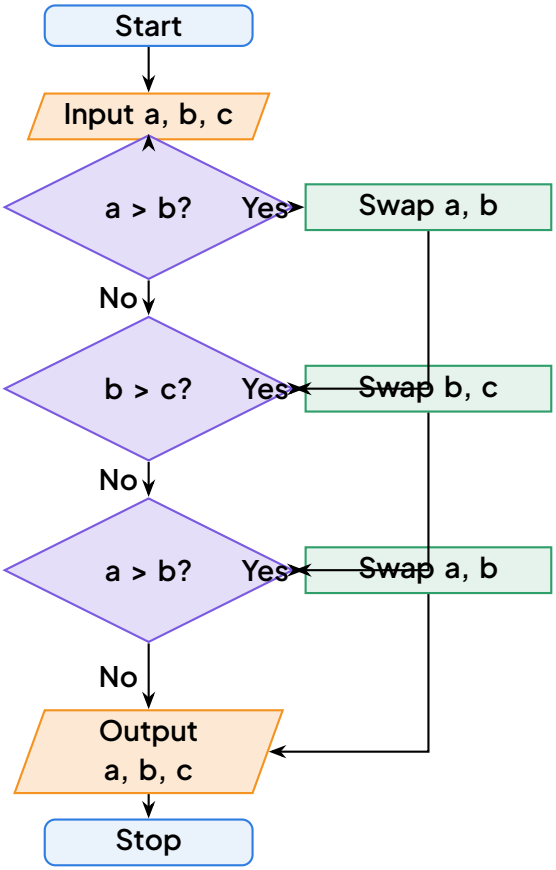
SOLUTION **Exercise 5 — Ascending order of three numbers**

Pseudocode:

```
READ a, b, c
IF a > b THEN
  temp <- a; a <- b; b <- temp
ENDIF
IF b > c THEN
  temp <- b; b <- c; c <- temp
ENDIF
IF a > b THEN
  temp <- a; a <- b; b <- temp
```

```
ENDIF
DISPLAY a, b, c
```

Flowchart:



Flowchart for sorting three numbers in ascending order

SOLUTION **Exercise 6 — Dry run of a WHILE loop**

1. Trace table:

n	Condition $n \leq 10$	p
2	True	$1 \times 2 = 2$
4	True	$2 \times 4 = 8$
6	True	$8 \times 6 = 48$
8	True	$48 \times 8 = 384$
10	True	$384 \times 10 = 3840$
12	False	(loop ends)

2. The algorithm displays 3840 (product of even numbers $2 \times 4 \times 6 \times 8 \times 10$).
3. The loop body is executed 5 times (for $n = 2, 4, 6, 8, 10$).

SOLUTION **Exercise 7 — From nested IF to CASE**

1. Rewritten algorithm using CASE:

```
READ m
CASE m DIV 10 OF
  0, 1, 2, 3, 4: grade <- "Fail"
  5, 6: grade <- "Pass"
  7: grade <- "Credit"
  8, 9, 10: grade <- "Distinction"
  OTHERWISE: grade <- "Invalid mark"
ENDCASE
DISPLAY grade
```

(Note: m div 10 gives the tens digit. Marks 0–49 give 0–4, 50–64 give 5–6, 70–79 give 7, 80–100 give 8–10. The `OTHERWISE` catches negative marks or marks > 100.)

2. Outputs:
   - $m = 45$: 45 div 10 = 4 → `Fail`
   - $m = 70$: 70 div 10 = 7 → `Credit`
   - $m = 95$: 95 div 10 = 9 → `Distinction`

SOLUTION **Exercise 8 — Hunting the sentinel error**

1. Dry run with input 60, 40, -1:

Step	*m* read	*s*	*n*
Initial	–	0	0
Read *m*	60	0	0
WHILE *m* ≠ -1 (60 ≠ -1 true)	–	–	–
Read *m*	40	–	–
s ← *s* + *m*	–	40	–
n ← *n* + 1	–	–	1
Loop: *m* = 40 ≠ -1 true	–	–	–
Read *m*	-1	–	–
s ← *s* + *m*	–	39	–
n ← *n* + 1	–	–	2
Loop: *m* = -1 ≠ -1 false → exit	–	–	–
Display *s*/*n*	–	39/2 = 19.5	–

Value displayed: 19.5 (incorrect; the true average of 60 and 40 is 50).

2. Two distinct errors:

   (a) The first mark (60) is read before the loop but never added to *s* nor counted in *n*; it is overwritten by the second `Read $m$` inside the loop.

   (b) The sentinel value -1 is read inside the loop, then added to *s* and counted in *n* before the condition is checked again. Thus -1 corrupts the sum and count.

3. Corrected algorithm:

```
s <- 0; n <- 0
READ m
WHILE m <> -1 DO
  s <- s + m
  n <- n + 1
  READ m
ENDWHILE
IF n > 0 THEN
  DISPLAY s / n
ELSE
  DISPLAY "No marks entered"
ENDIF
```

Now the first mark is processed, the sentinel is not added, and no mark is lost.

SOLUTION **Exercise 9 — Multiplication table — FOR and WHILE**

1. Python program (FOR loop):

```
# Multiplication table using FOR loop
N = int(input("Enter an integer N: "))
print(f"Multiplication table of {N}:")
for i in range(1, 13):
  print(f"{N} x {i} = {N*i}")
```

2. Modified program (WHILE loop with user-defined limit L):

```
# Multiplication table using WHILE loop
N = int(input("Enter an integer N: "))
L = int(input("Enter upper limit L: "))
print(f"Multiplication table of {N} up to {L}:")
i = 1
while i <= L:
  print(f"{N} x {i} = {N*i}")
  i = i + 1
```

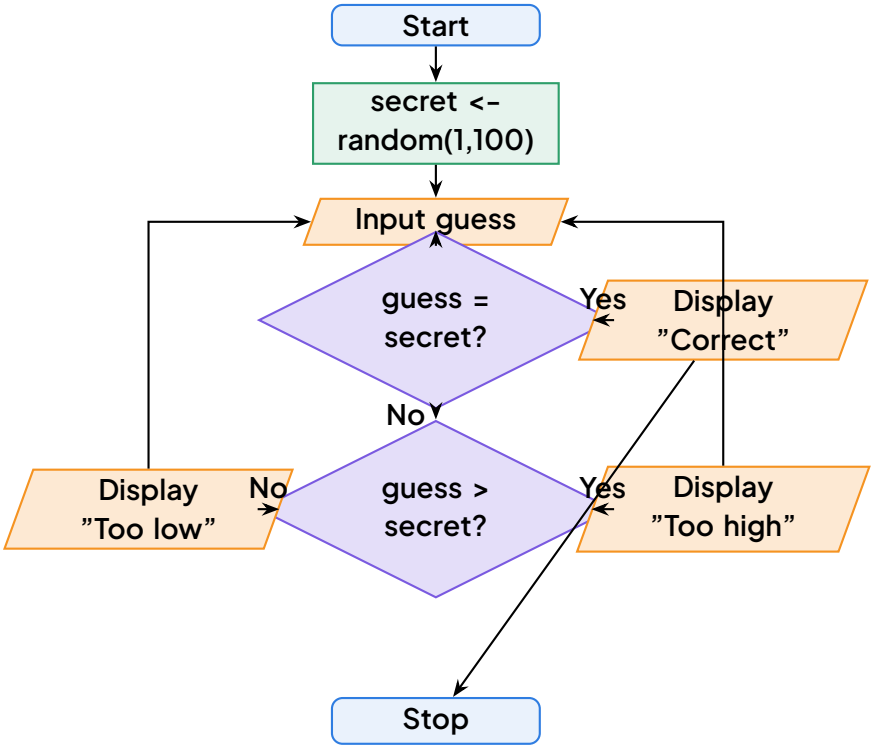
3. Advantage of FOR loop: The `FOR` loop automatically handles the initialization, condition testing, and increment of the counter in a single line, making the code more concise, less error-prone (no risk of forgetting to increment the counter), and clearer when the number of iterations is known in advance.

Mark scheme (indicative):

- (a) 4 marks: correct input (1), correct FOR loop syntax (1), correct range 1..12 (1), correct output format (1).
- (b) 3 marks: input L (1), correct WHILE loop with condition $i \leq L$ (1), correct increment inside loop (1).
- (c) 1 mark: any valid advantage (conciseness, automatic counter management, readability).

SOLUTION **Exercise 10 — Number-guessing game**

1. Flowchart:



Flowchart for number-guessing game

2. Pseudocode (using REPEAT … UNTIL):

```
secret <- RANDOM(1, 100)
REPEAT
  READ guess
  IF guess > secret THEN
    DISPLAY "Too high"
  ELSE IF guess < secret THEN
    DISPLAY "Too low"
  ELSE
    DISPLAY "Correct"
  ENDIF
UNTIL guess = secret
```

3. Dry run with secret = 37, attempts: 50, 25, 37:
   - Guess 50: 50 > 37 → `Too high`
   - Guess 25: 25 < 37 → `Too low`
   - Guess 37: 37 = 37 → `Correct` (loop ends)

   Sequence of messages: `Too high`, `Too low`, `Correct`.

Mark scheme (indicative):

- (a) 5 marks: correct symbols (1), correct sequence: start, generate secret, input, decision (guess=secret?), two branches for high/low, loop back, stop on correct (4).
- (b) 3 marks: secret generation (1), loop with condition (1), correct selection with three branches (1).
- (c) 2 marks: correct three messages in order (2).

SOLUTION **Exercise 11 — Class marks statistics — GCE style**

1. Purpose of variables:
   - `pass`: counts the number of students with mark ≥ 50.
   - `fail`: counts the number of students with mark < 50.
   - `max`: stores the highest mark encountered so far.
   - `sum`: accumulates the total of all marks to compute the average.
2. The loop runs from 1 to 30 inclusive: 30 times.
3. Trace table for first three marks (45, 72, 50), loop stops after 3, avg = sum/3:

i	*m*	*pass*	*fail*	*max*	*sum*	*avg* (after loop)
Start	–	0	0	0	0	–
1	45	0	1	45	45	–
2	72	1	1	72	117	–
3	50	2	1	72	167	55.67

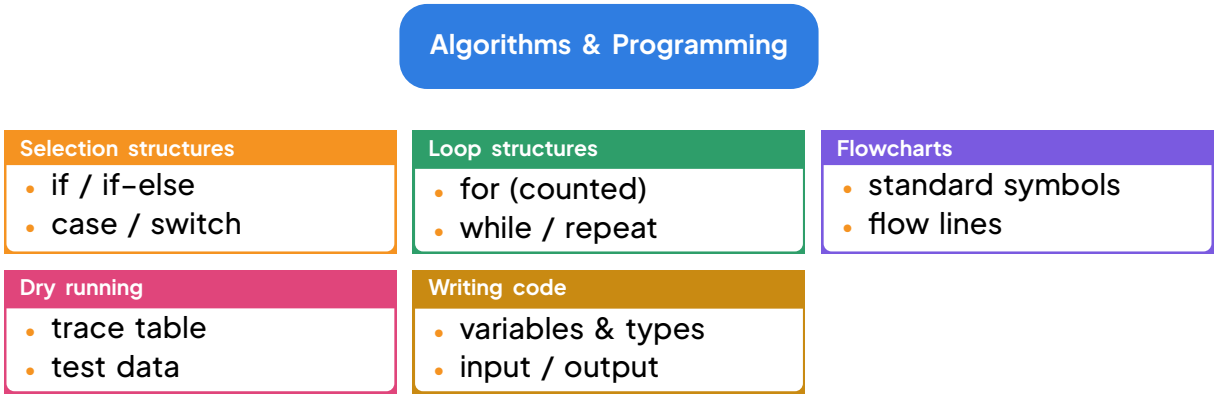
   Values displayed: $pass = 2$, $fail = 1$, $max = 72$, $avg = 55.67$ (or $167/3$).
4. Problem with $max \leftarrow 0$: If all marks are negative (e.g., a marking scheme with penalties) or if the pass mark is above 0 but all students score below 0, max would remain 0, which is not a real mark. Even with non-negative marks, if a student scores 0, max would correctly become 0, but initialising to 0 assumes marks are ≥ 0. A safer initialisation is to set max to the first mark read (or to a very small number like -1 if marks are non-negative). Correction: Read the first mark before the loop, set $max \leftarrow m$, $sum \leftarrow m$, update $pass/fail$, then loop for the remaining 29 students.
5. Control structures used:
   - `FOR` loop (counter-controlled iteration) — loop control structure.
   - `IF … THEN … ELSE … ENDIF` (two instances) — selection control structure.

Error classification: The error in (d) is a logic error. The algorithm runs without crashing (no runtime error) and the syntax is correct (no syntax error), but it produces an incorrect result for certain valid inputs (all marks < 0). The algorithm does not correctly implement the specification "find the maximum mark" for all possible data sets.

Mark scheme (indicative):

- (a) 2 marks: 0.5 each for correct purpose of pass, fail, max, sum.
- (b) 1 mark: 30.
- (c) 4 marks: trace table with correct columns (1), correct values for each step (2), correct final displayed values (1).
- (d) 2 marks: explanation of why 0 is problematic (1), valid correction (1).
- (e) 3 marks: identify FOR and IF (1), classify as logic error with justification (2).

11 SUMMARY SHEET



Mind map of the chapter: the five pillars of algorithmic thinking.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- An **algorithm** is a finite, ordered sequence of unambiguous steps that solves a problem.
- A **control structure** determines the order in which instructions are executed: sequence, selection, iteration.
- **Selection** (decision) executes a block only if a condition is true: `IF condition THEN ... ELSE ... ENDIF`; multiple choices use `CASE ... OF`.
- A **loop** repeats a block: `FOR` (number of repetitions known in advance), `WHILE` (test before each pass, may execute 0 times), `REPEAT ... UNTIL` (test after, executes at least once).
- A **flowchart** is a diagram of an algorithm using standard symbols: oval (start/stop), parallelogram (input/output), rectangle (process), diamond (decision), arrows (flow).
- **Dry running** means executing an algorithm by hand with test data, recording variable values in a **trace table**.

Methods to master

- Designing a solution: understand the problem → identify inputs, processing, outputs → write the algorithm in pseudocode → draw the flowchart → dry run → translate into code.
- Choosing a loop: known number of repetitions ⇒ `FOR`; unknown, condition checked

first ⇒ `WHILE`; body must run at least once ⇒ `REPEAT ... UNTIL`.

- Trace table: one column per variable and one for output; one row per executed step; update only what changes.
- Writing code: declare variables with correct types (integer, real, string, Boolean), read inputs, process with selection/loops, display results with clear messages.

Common mistakes

- Infinite loops: forgetting to update the loop control variable inside `WHILE` or `REPEAT`.
- Off-by-one errors: wrong initial value or wrong bound in a `FOR` loop.
- Using = (assignment) instead of the comparison operator in a condition.
- Missing `ENDIF` / `ENDWHILE`, or a decision diamond without two labelled branches (Yes/No) in a flowchart.
- Swapping symbols: parallelogram for processing or rectangle for input/output.
- Dry running only one data set: always test normal, boundary and invalid data.

GCE tips

- Read the question twice: many marks are lost by answering with code when a flowchart (or pseudocode) is requested.
- In a trace table, show every intermediate value; examiners award method marks line by line.
- Label decision branches clearly and make sure every flowchart path ends at a Stop symbol.
- State the data type of each variable you declare; initialise counters and accumulators (e.g. `sum $\leftarrow$ 0`) before the loop.
- Manage time: a dry-run question is quick marks if your table is neat and ruled.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+